

Deep Learning

Session 6: Sequence methods and time-series analysis

October 8, 2024

This session will be recorded for in-class use.

Student and Alumni Advisory Board

- Independent body that links the Centre with the students and alumni
- SAAB organises student- and alumni-led events and provides advice on programming, events and other activities hosted by the Centre
- 6-8 Board members serve for one academic year

As a member you will...

- Actively contribute to the Centre's initiatives
- Engage in discussions with the Centre team and share insights and perspectives
- Help shape the future student- and alumni facing activities of the Centre

Learn more and apply now!



Midterm

- Date and time: Tuesday, October 30, 2024, at 14:00
- Length: 90 minutes
- Bring a pen, 1 cheat sheet and ID
- No calculator
- Material covered:
 - Lectures 1-6
 - Problem sets (emphasis on theoretical part but also conceptual things from the practical part)
 - Disclaimer: This class builds on the ML class and Maths for Data Science. We expect you to have a working knowledge of important concepts covered there that are needed for deep learning.

Midterm

- Study guide:
 - Recap Sessions 1-6
 - Recap the problem set solutions and practice modifications
 - We'll provide few practice problems
 - Zhang et al. textbook & exercises (more advanced)
- Types of problems:
 - Midterm is similar to DL problem sets and practice problems
 - True/False problems, multiple choice possible
 - Mathematical derivations or calculations as solved in the lectures and problem sets
 - Conceptual questions asking you to describe something in words and/or draw graphs

Problem set

- Problem Set 2 will be out today
- New groups already assigned

Midterm course feedback

- Provide feedback on Moodle this week
- Helps us determine how it is going and how to adjust

Deep learning theory

1. Deep learning in public policy
2. Deep neural networks (1)
3. Deep neural networks (2)

Applications

4. Computer vision (1): Convolutional neural networks
5. Computer vision (2): Modern CNNs, implementation, and applications
6. **Sequence methods and time-series analysis**
7. Natural language processing (1)
8. Natural language processing (2): Transformer models, implementation, and applications
- Midterm exam -
9. Further topics in deep learning

Deep learning and public policy

10. Policy approaches to regulate deep learning and responsible implementation in government
11. Deep learning in practice
12. Tutorial presentations

- Motivating sequences and time series in public policy
- Overview of sequence modeling tasks and challenges
- Basic building blocks that comprise state-of-the-art sequence models
 - Recurrent Neural Networks (RNN)
 - Long Short Term Memory (LSTM) and Gated Recurrent Unit (GRU)
 - Temporal Convolutions
- Example Task: Time series forecasting

Slides are based on a guest lecture by Marcus Voss in Fall 2022.

Introduction to Sequence Modeling

Sequential Data

Images

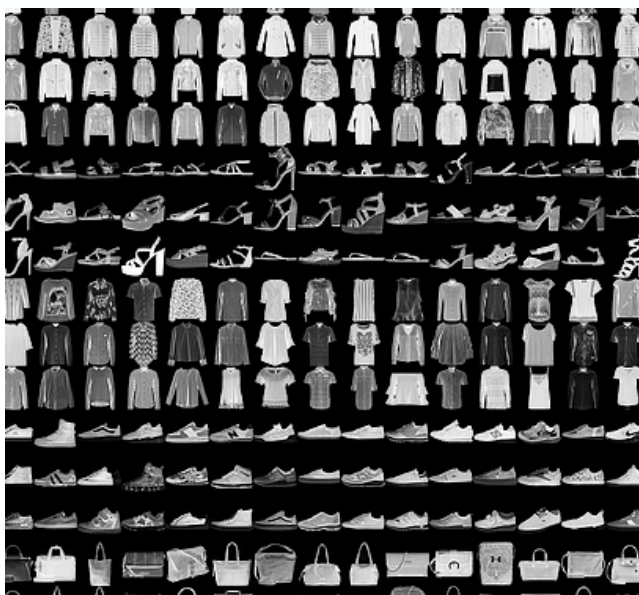


Image: Fashion MNIST

Tabular data

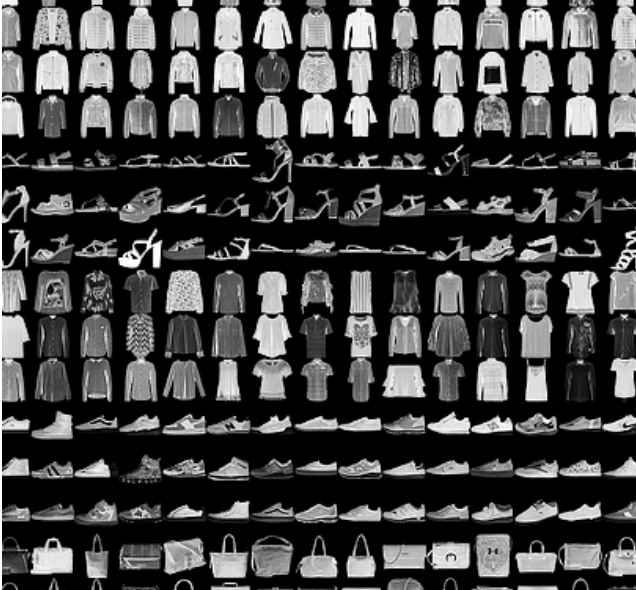
Data summary

Carbon emissions within each local authority in the UK Click heading to sort. Figures in KT CO2							
Local auth.	Region	Indst & Comm	Domestic	Road Trans- port	Land use, land-use change & forestry	Total	Per Capita, tonnes
Darlington	NE	335.34	274.26	213.88	5.63	829.12	8.3
Chester-le-Street	NE	65.47	129.55	109.06	0.5	304.58	5.7
Derwentside	NE	190.49	252.54	126.57	-1.54	568.06	6.4
Durham	NE	250.99	220.6	219.19	1.97	692.75	7.4
Easington	NE	241.61	255.88	189.17	3.16	689.81	7.2
Sedgefield	NE	518.18	234.12	206.92	4.09	963.31	11.1

Image: Tony Hirst, Carbon emissions data table from flickr

Sequential Data

Images



Tabular data

Data summary

Carbon emissions within each local authority in the UK
Click heading to sort. Figures in KT CO2

Local auth.	Region	Indst & Comm	Domestic	Road Trans- port	Land use, land-use change & forestry	Total	Per Capita, tonnes
Darlington	NE	335.34	274.26	213.88	5.63	829.12	8.3
Chester-le-Street	NE	65.47	129.55	109.06	0.5	304.58	5.7
Derwentside	NE	190.49	252.54	126.57	-1.54	568.06	6.4
Durham	NE	250.99	220.6	219.19	1.97	692.75	7.4
Easington	NE	241.61	255.88	189.17	3.16	689.81	7.2
Sedgefield	NE	518.18	234.12	206.92	4.09	963.31	11.1

Observations for non-sequential data:

1. Order of instances within the dataset does not matter
2. Values of one instance do not depend on values of another
3. Same size of each of the instances

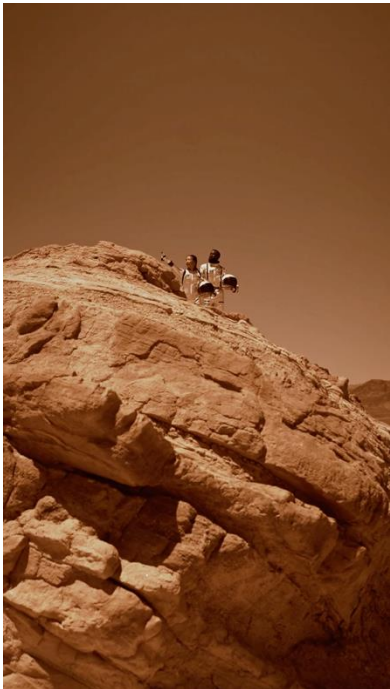
Sequential Data

Sequential Data: Data is characterized by: **i)** that the ordering of instances is relevant and **ii)** that instances depend on other instances in the dataset.

Can you think of sequential data?

Sequential Data

Video



Sound

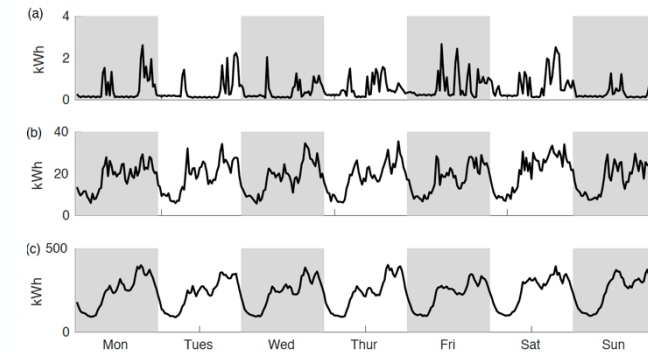


Language

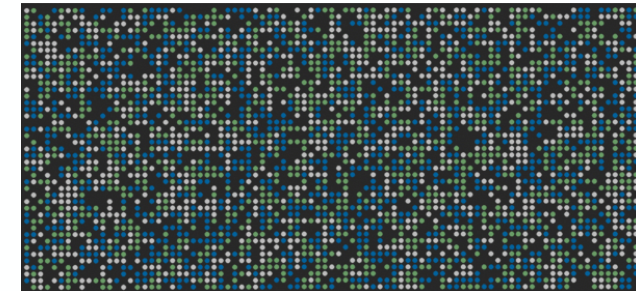
I am a highly intelligent question answering bot. If you ask me a question that is rooted in truth, I will

A: provide a truthful answer.

Time Series



Other



Sequential Data in Public Policy - Video

Political Communications



Political Debate



News



Sequences in Public Policy - Sound

Newly Released Tapes Show Nixon Maneuvering as Watergate Unfolds

 Give this article    42



President Richard M. Nixon on April 29, 1974, after he announced that he would turn over transcripts of the White House tapes to House impeachment investigators.
Associated Press

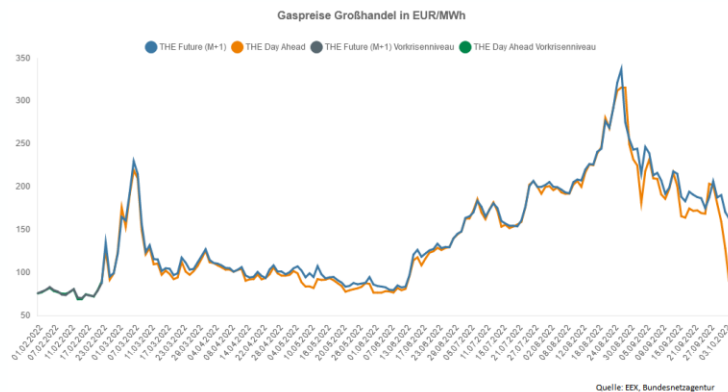
Laws (and law proposals)



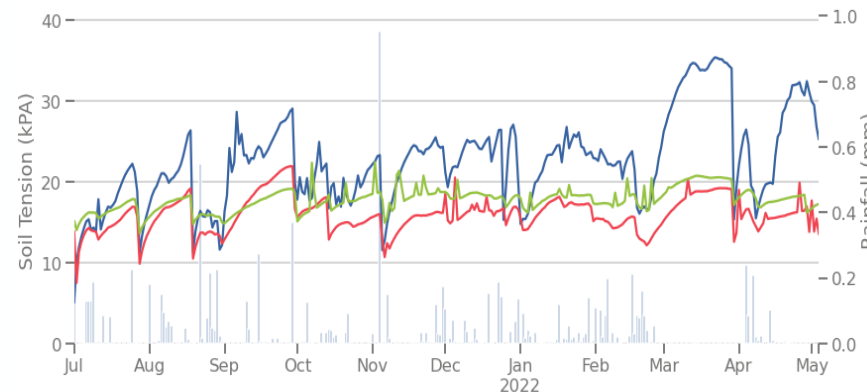
Sequences in Public Policy – Time Series

Time series is sequential data that is *indexed by time*. Often (but not necessarily), time series are successive *equally spaced* data points indexed by time.

Market prices



Sensor Values



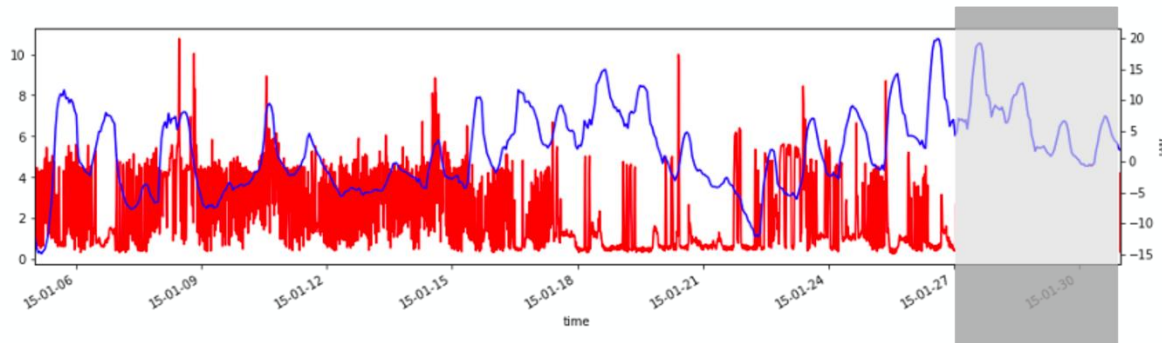
Log Files

```
(Sun Sep 13 23:02:05 2009): Beginning Wbemupgd.dll Registration
(Sun Sep 13 23:02:05 2009): Current build of wbemupgd.dll is 5.1.2600.2180 (
xpsp_sp2_rtm.040803-2158)
(Sun Sep 13 23:02:05 2009): Beginning Core Upgrade
(Sun Sep 13 23:02:05 2009): Beginning MOF load
(Sun Sep 13 23:02:05 2009): Processing C:\WINDOWS\system32\WBEM\cimwin32.mof
(Sun Sep 13 23:02:09 2009): Processing C:\WINDOWS\system32\WBEM\cimwin32.mfl
(Sun Sep 13 23:02:12 2009): Processing C:\WINDOWS\system32\WBEM\system.mof
(Sun Sep 13 23:02:16 2009): Processing C:\WINDOWS\system32\WBEM\evntprv.mof
(Sun Sep 13 23:02:16 2009): Processing C:\WINDOWS\system32\WBEM\hnetcfg.mof
(Sun Sep 13 23:02:16 2009): Processing C:\WINDOWS\system32\WBEM\sr.mof
(Sun Sep 13 23:02:16 2009): Processing C:\WINDOWS\system32\WBEM\dgnet.mof
(Sun Sep 13 23:02:16 2009): Processing C:\WINDOWS\system32\WBEM\whqlprov.mof
(Sun Sep 13 23:02:16 2009): Processing C:\WINDOWS\system32\WBEM\ieinfo5.mof
(Sun Sep 13 23:02:17 2009): MOF load completed.
(Sun Sep 13 23:02:17 2009): Beginning MOF load
(Sun Sep 13 23:02:17 2009): MOF load completed.
(Sun Sep 13 23:02:17 2009): Core Upgrade completed.
(Sun Sep 13 23:02:17 2009): Wbemupgd.dll Service Security upgrade succeeded.
(Sun Sep 13 23:02:17 2009): Beginning WMI(WDM) Namespace Init
(Sun Sep 13 23:02:20 2009): WMI(WDM) Namespace Init Completed
(Sun Sep 13 23:02:20 2009): ESS enabled
(Sun Sep 13 23:02:20 2009): ODBC Driver <system32>\wbemdr32.dll not present
(Sun Sep 13 23:02:20 2009): Successfully verified WBEM ODBC adapter (
incompatible version removed if it was detected).
(Sun Sep 13 23:02:20 2009): Wbemupgd.dll Registration completed.
(Sun Sep 13 23:02:20 2009):
```

Examples of sequence modeling tasks

Forecasting and predicting next steps of a sequence

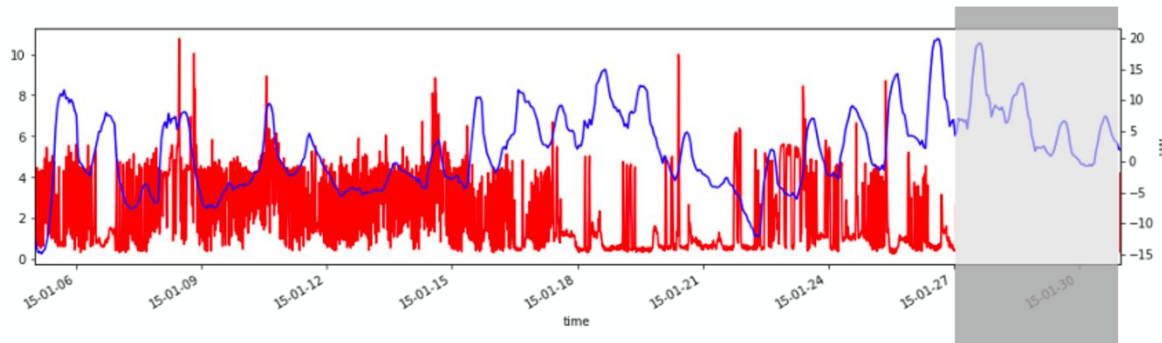
Electric Load Forecasting



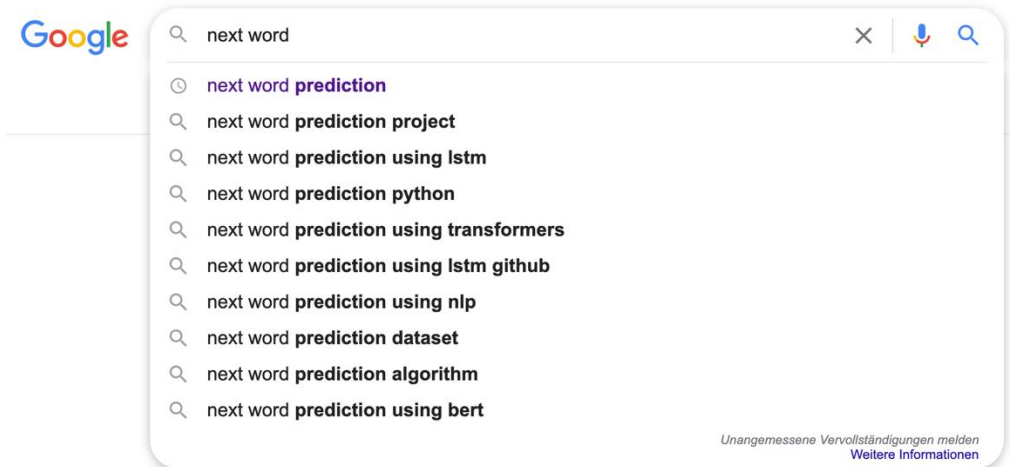
Examples of sequence modeling tasks

Forecasting and predicting next steps of a sequence

Electric Load Forecasting



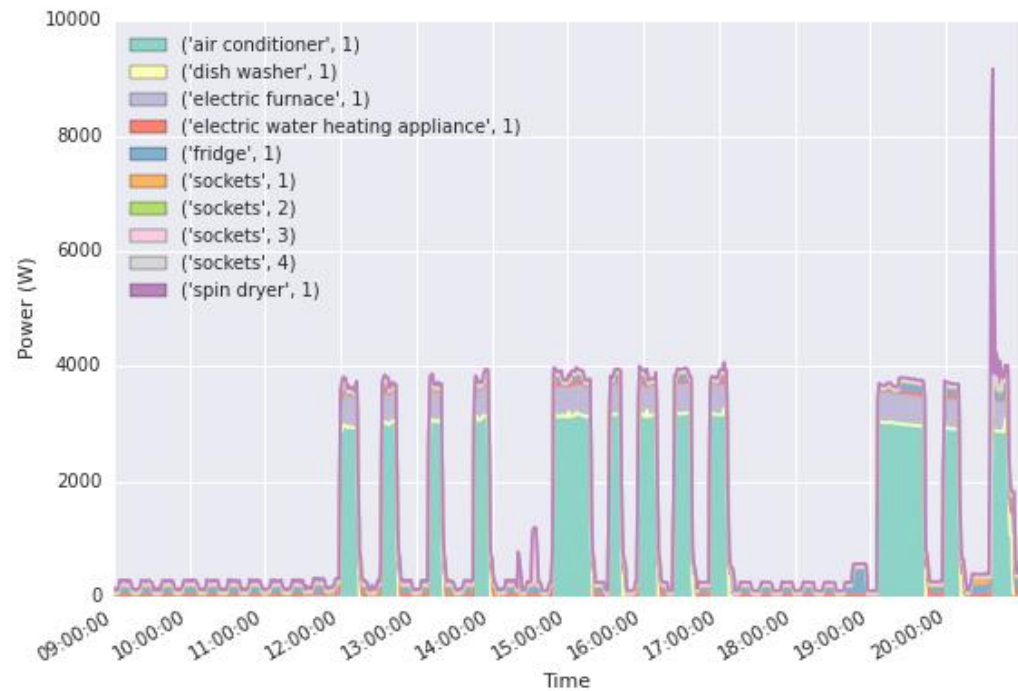
Search Query Completion



Examples of sequence modeling tasks

Classification

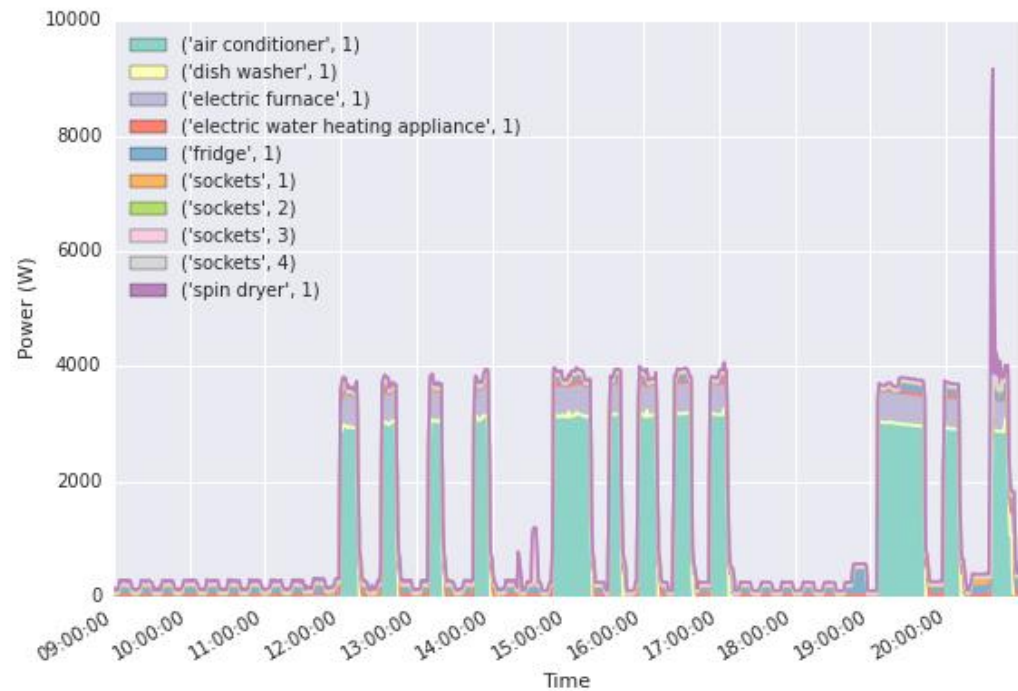
Non-Intrusive Load Monitoring



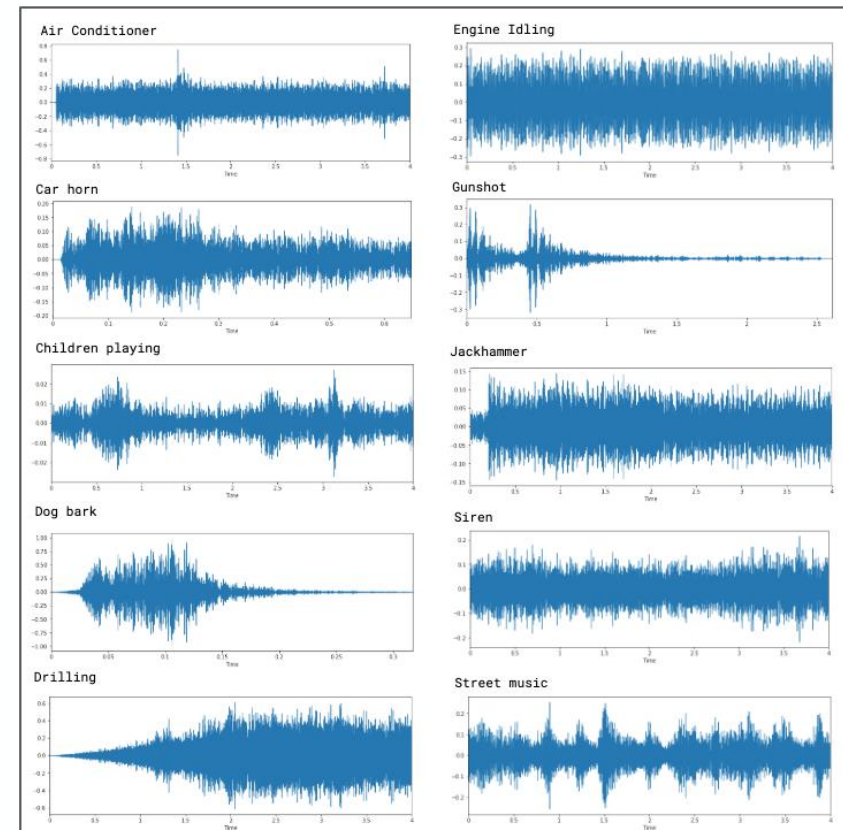
Examples of sequence modeling tasks

Classification

Non-Intrusive Load Monitoring



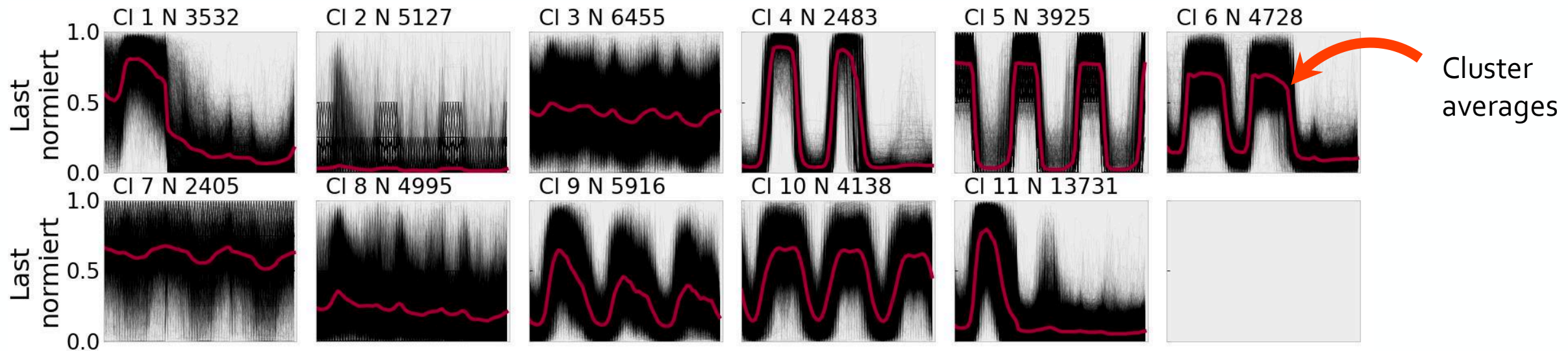
Classifying Sound



Examples of sequence modeling tasks

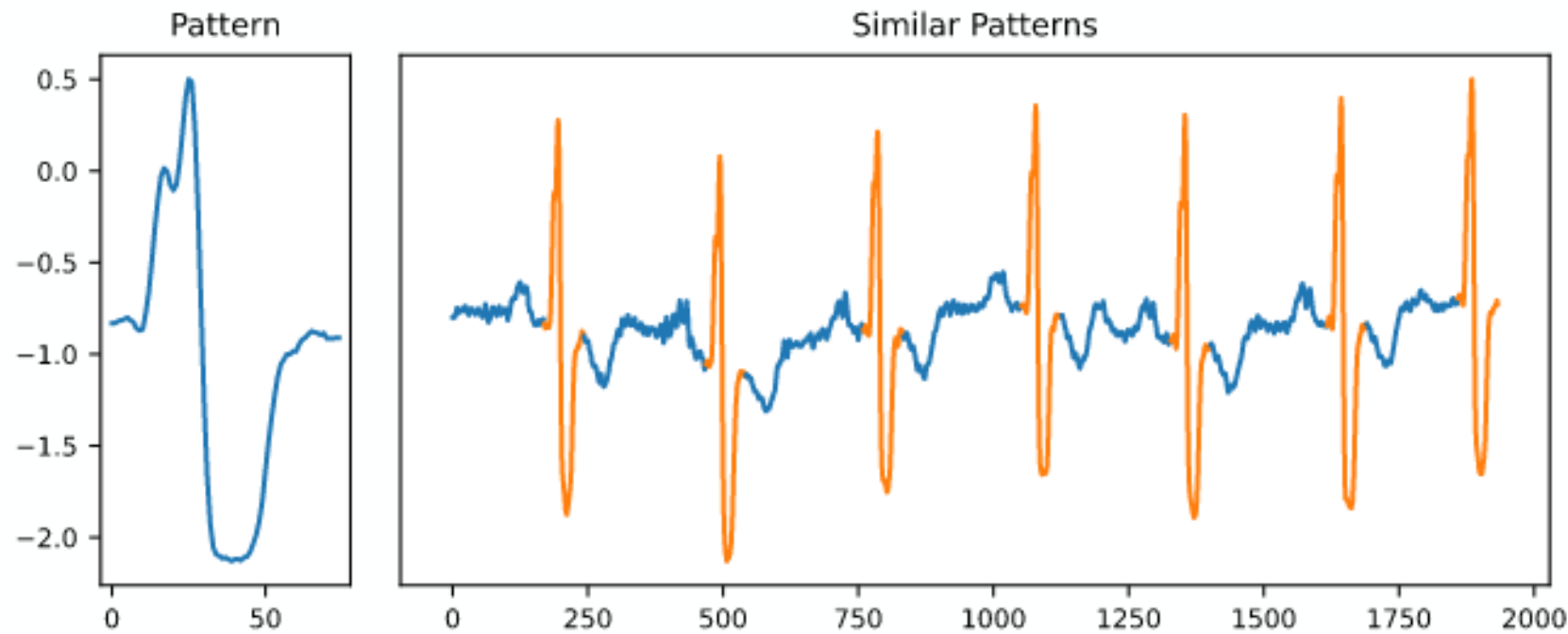
Clustering

Example: 11 Clusters of load profiles of industrial customers determined by k-Means clustering.



Examples of sequence modeling tasks

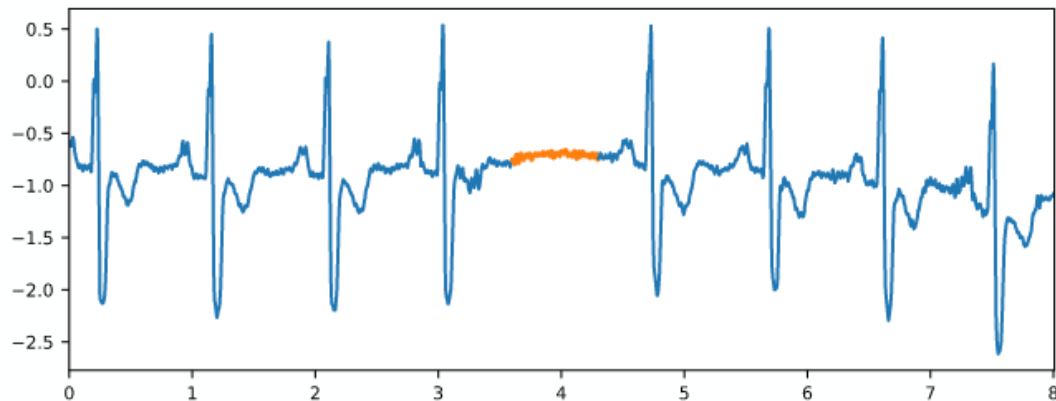
Pattern Matching: Finding („querying“) a known pattern within a sequence.



Examples of sequence modeling tasks

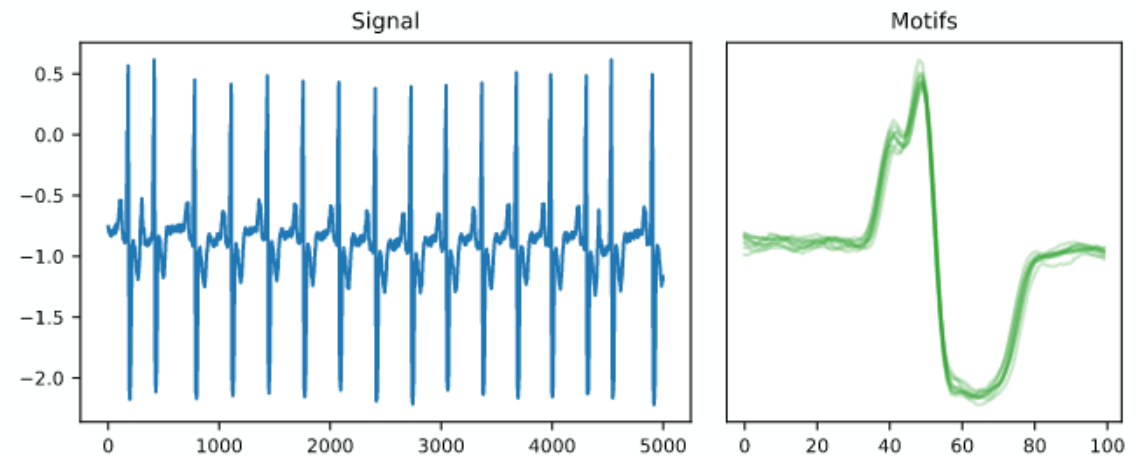
Anomaly Detection

The task of finding abnormal data points („outliers“) or subsequences („discords“). Applications e.g., in predictive maintenance.



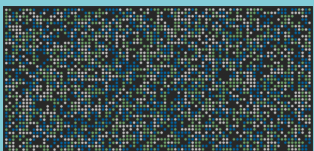
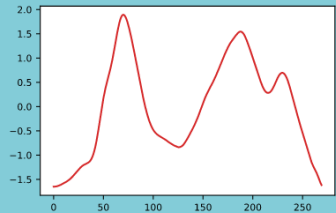
Motif Detection

The task of finding time series subsequences that appears recurrently. Applications e.g., in analyzing DNA sequences.



General approaches to sequence modeling tasks

Sequences



Raw sequence

Deep learning

Features

Manual

Automated

Feature selection algorithms,
dimensionality reduction, **neural
net embeddings**, ...

Modeling Task

Forecasting

Classification

Clustering

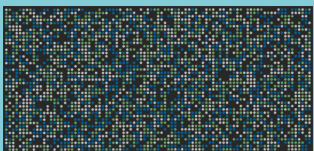
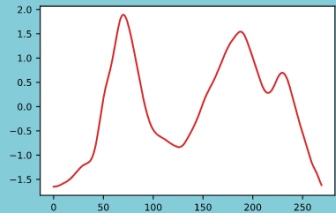
Anomaly Detection

Pattern Recognition

...

General approaches to sequence modeling tasks

Sequences



Raw sequence

Features

Manual

Automated

Feature selection algorithms,
dimensionality reduction, **neural
net embeddings**, ...

Modeling Task

Forecasting

Classification

Clustering

Anomaly Detection

Pattern Recognition

...

Feature Modeling for Sequences – Example Text

Example: Bag-of-Words

The cat sat.

The cat sat in the hat.

The cat with the hat.



Vocabulary:

the, cat, sat, in, hat, and with.



Count to vectorize

The cat sat: [1, 1, 1, 0, 0, 0]

The cat sat in the hat: [2, 1, 1, 1, 1, 0]

The cat with the hat: [2, 1, 0, 0, 1, 1]

Weakness:

Order not preserved, and can change meaning drastically

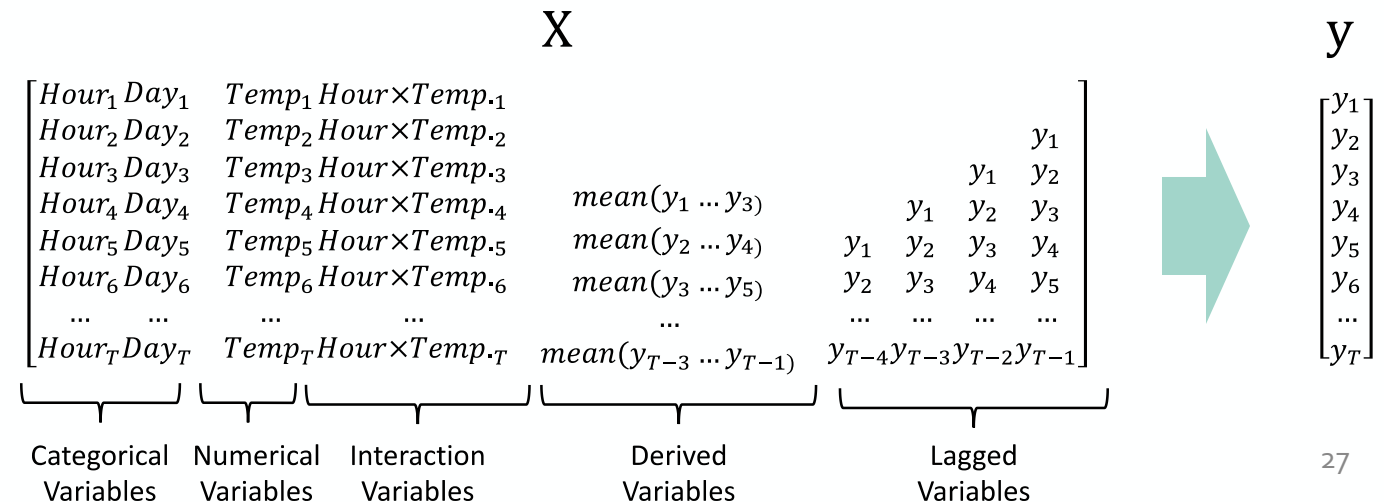
One Solution: n-grams -> explosion of vocabulary size!

Feature Modeling for Sequences – Example Time Series

Example: Manual features using domain knowledge in electric load forecasting

In load forecasting the literature the most common features are:

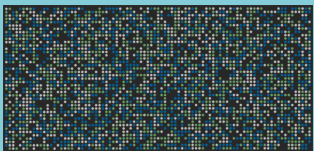
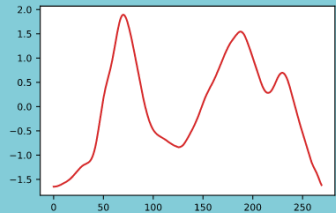
- weather-related **external variables** (temperature, humidity, solar irradiance),
- Daily, weekly and annual **seasonality**,
- Load of the last hours („**lagged values**“)
- Socioeconomic indicators (number of residents, demographics, floor space, tariffs and other interventions, monthly energy consumption)



Illustrative representation of features for load forecasting as machine learning problem.

General approaches to sequence modeling tasks

Sequences



Vision: Modeling the task end-to-end

Raw Sequence

Features

Manual

Automated

Feature selection algorithms,
dimensionality reduction, **neural
net embeddings**, ...

Modeling Task

Forecasting

Classification

Clustering

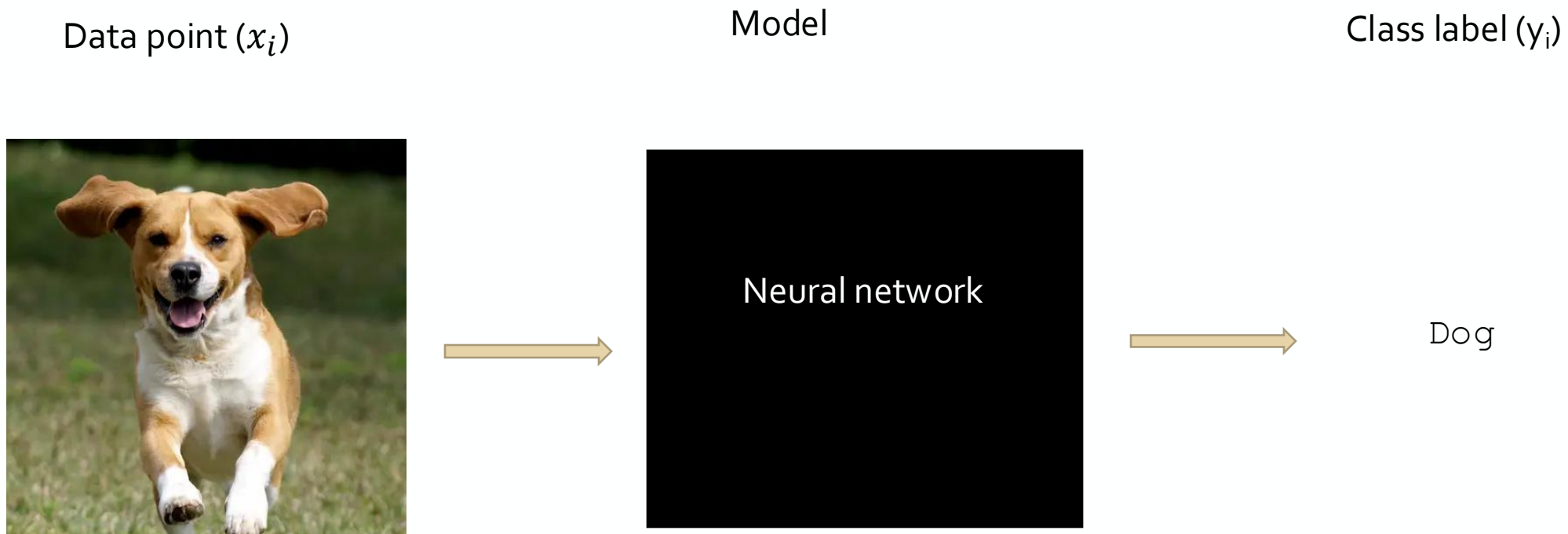
Anomaly Detection

Pattern Recognition

...

Raw Sequence modelling – why is it hard?

Recall from Session 2: Machine Learning



Raw Sequence modelling – why is it hard?

Machine Learning is function learning

$$f(x) = \hat{y}$$

Model

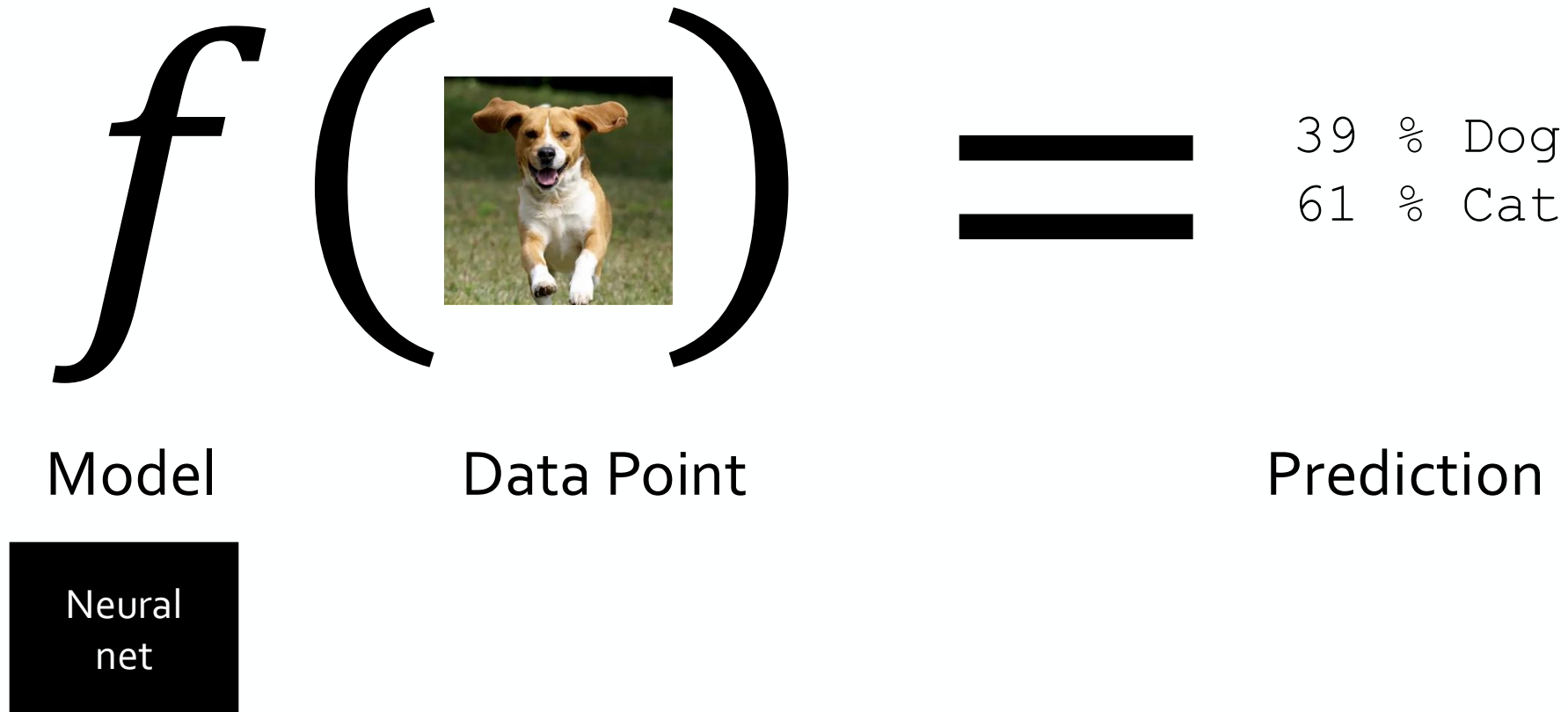
Data Point

Prediction

Neural
net

Raw Sequence modelling – why is it hard?

Machine Learning is function learning



Raw Sequence modelling – why is it hard?

Machine Learning is function learning



Model

Data Point

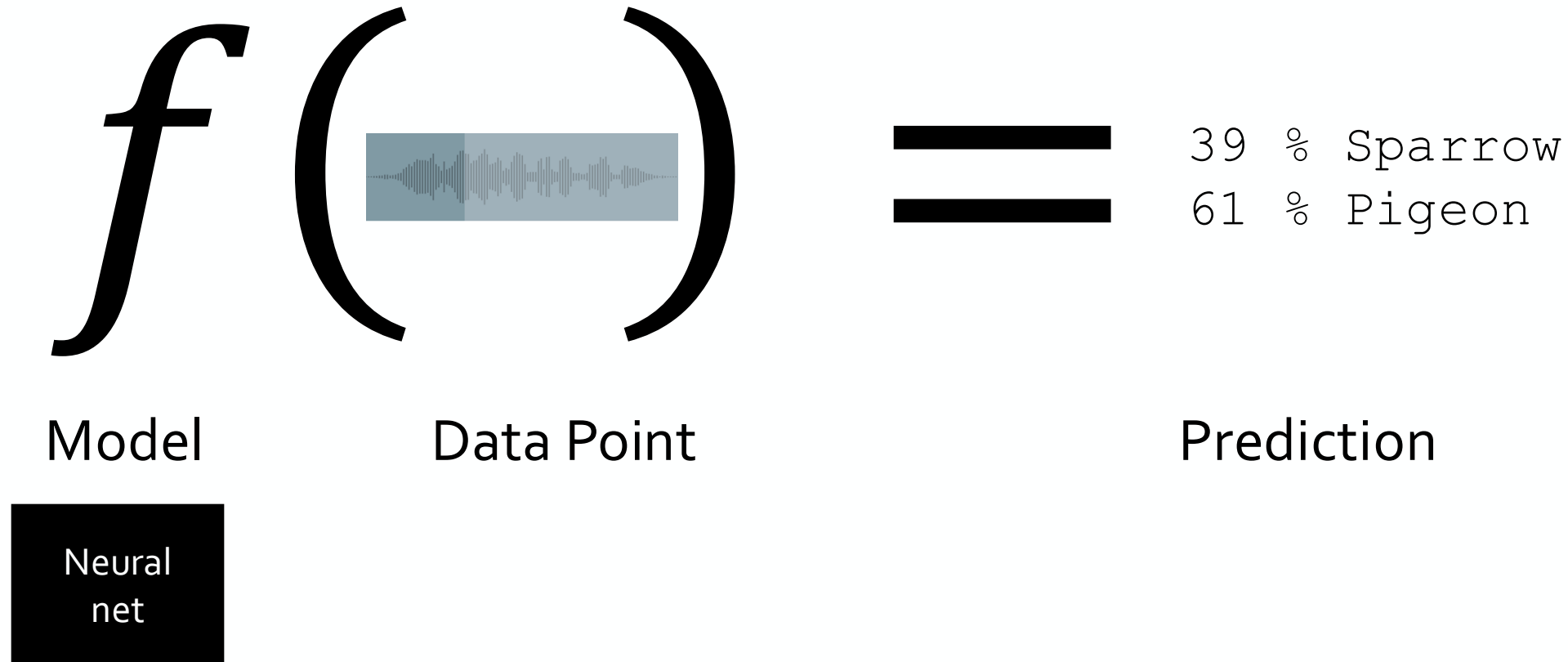
39 %
Advertisement
61 % Scientific
Report

Prediction

Neural
net

Raw Sequence modelling – why is it hard?

Machine Learning is function learning



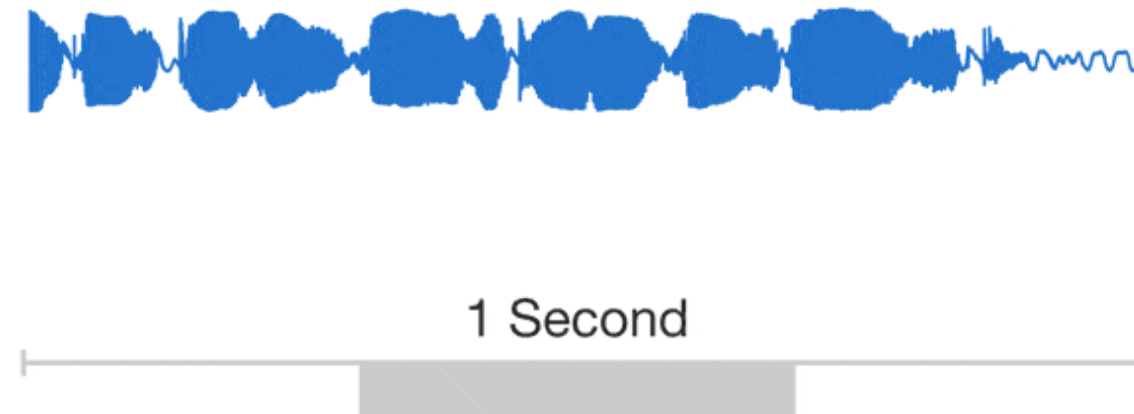
Raw Sequence modelling – why is it hard?

Challenge 1: Most (standard) machine learning models require fixed size input and output!

Challenge 2: For some sequences, dependencies exist at very different time scales.

Fixed size vector


$$f: \mathbb{R}^d \mapsto \mathbb{R}$$

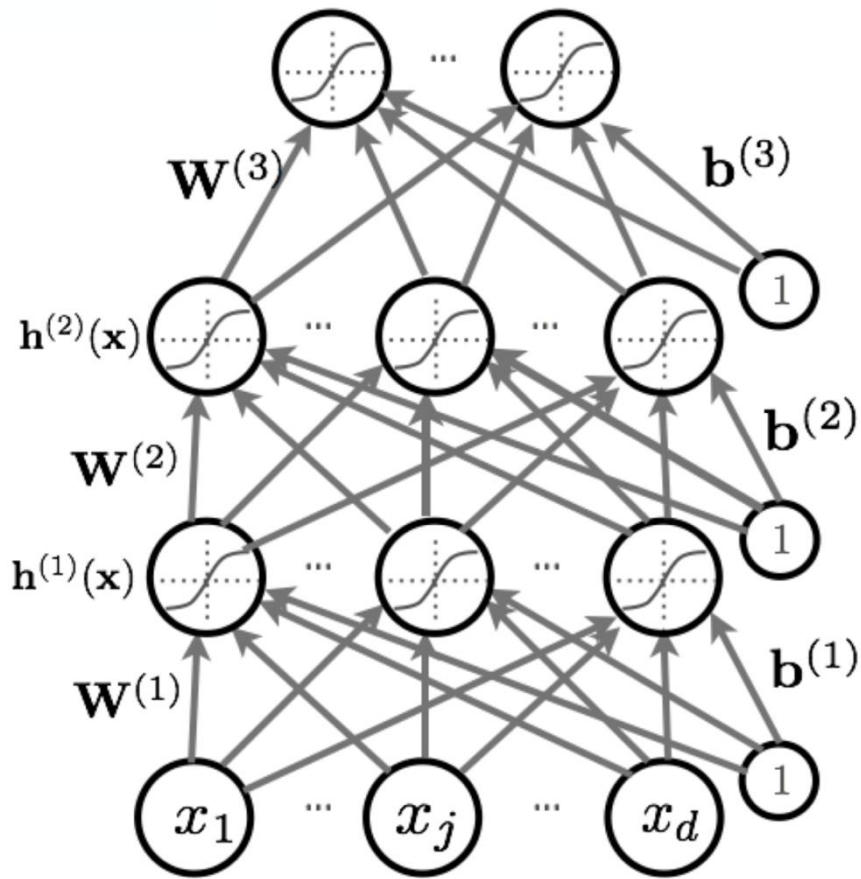


Sequence Models

Overview of Sequence Models

1. „Vanilla“ Recurrent Neural Networks (RNN)
2. Long Short Term Memory (LSTM) and Gated Recurrent Units (GRU)
3. Convolutional Neural Networks for Sequences

Recurrent Neural Networks



- So far we only worked with “fully connected” layers and convolutional layers
- Fully connected: Every node is connected to every node of the next layer
- Fully connected neural networks compute $f(x_1, x_2, \dots, x_d)$ for a fixed dimension d .

How can we compute $f(x_1, x_2, \dots, x_N)$ for an N that may vary?

How can we ensure that between the inputs there is dependence?

Recurrent Neural Networks

How can we compute $f(x_1, x_2, \dots, x_N)$ for variable N ?

We calculate can calculate it *recurrently*:

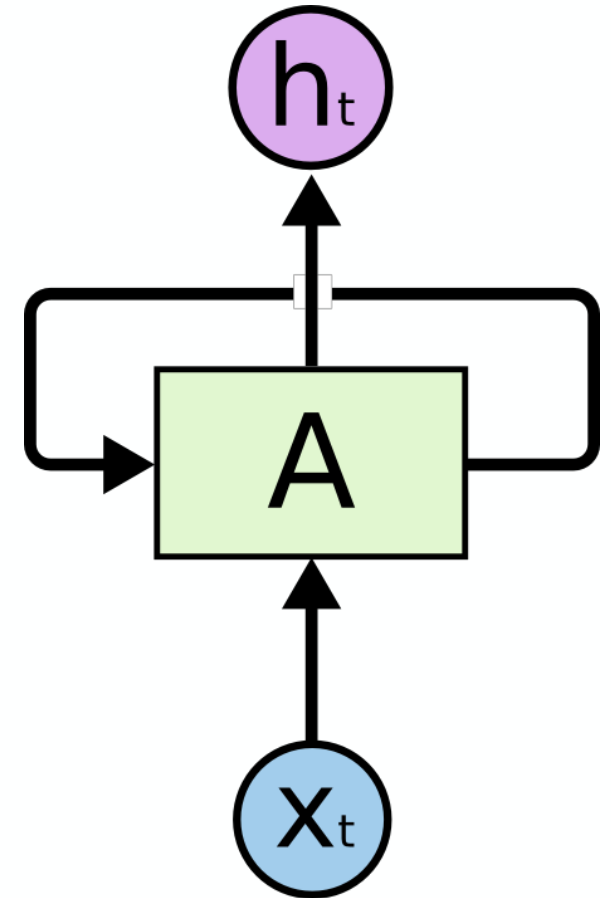
$$h_t = A(h_{t-1}, x_t), \quad \text{for } i = 1, \dots, N$$

Final prediction is then the output of the final calculation:

$$f(x_1, x_2, \dots, x_N) = h_N$$

where

- A is a neural network or a neural network unit.
- h_t is an intermediate hidden state.



Recurrent Neural Networks

Recurrent neural networks (RNN) can be thought of as multiple applications of the same network at different time steps, each passing an activation to a successor. This makes RNN „deep“ neural networks, even if technically only one layer is modeled.

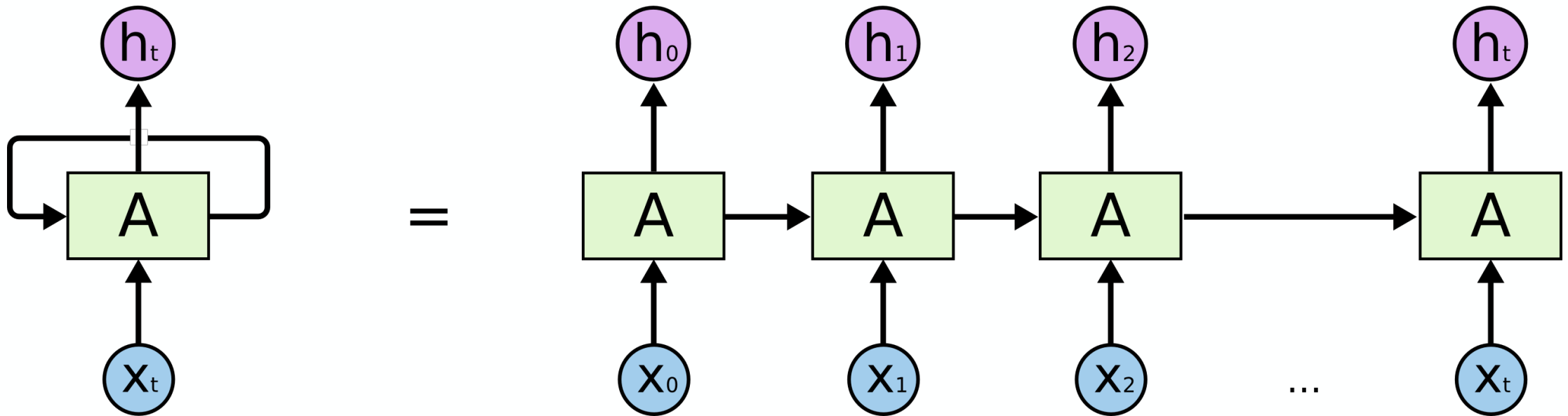


Illustration of an “unrolled” RNN.

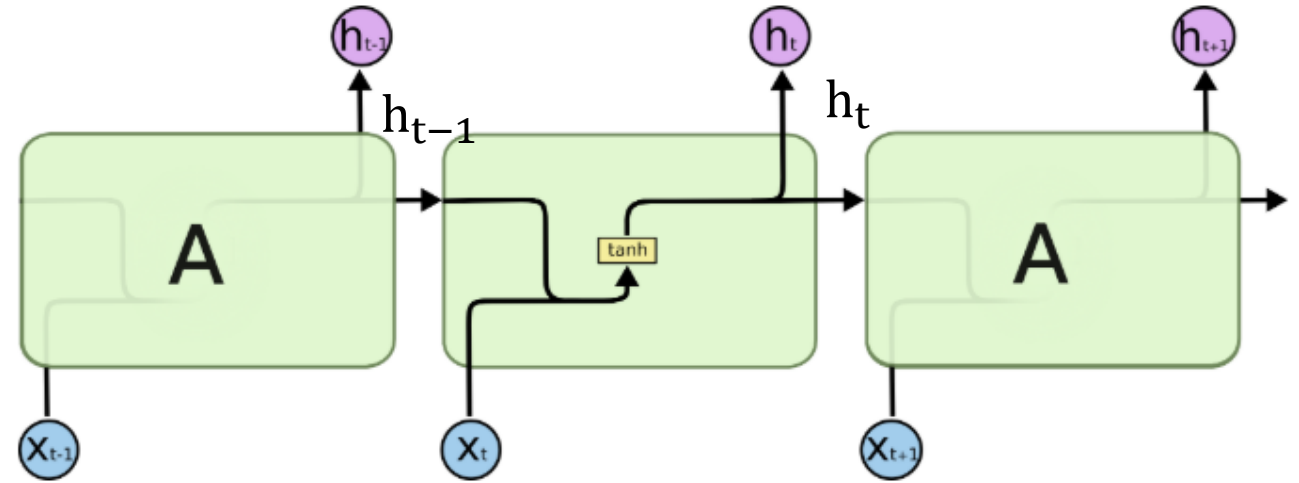
“Vanilla” Recurrent Neural Networks



If A is only an activation function (typically $\tanh$), then these RNN are also called “Vanilla” Recurrent Neural Networks or standard RNN.

$h_t = A(h_{t-1}, x_t)$ is then:

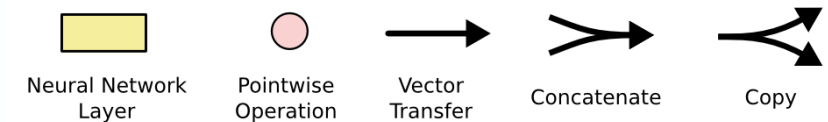
$$h_t = \tanh(W \cdot [h_{t-1}, x_t] + b)$$



Detailed illustration of the unit in an “unrolled” RNN.

where

- W is the weight vector,
- b is a bias term,
- $[\bullet]$ means concatenation of the vectors.



Recurrent Neural Networks

Partner discussion

How many weight matrices W and b do we have? What is the dimension of W ?

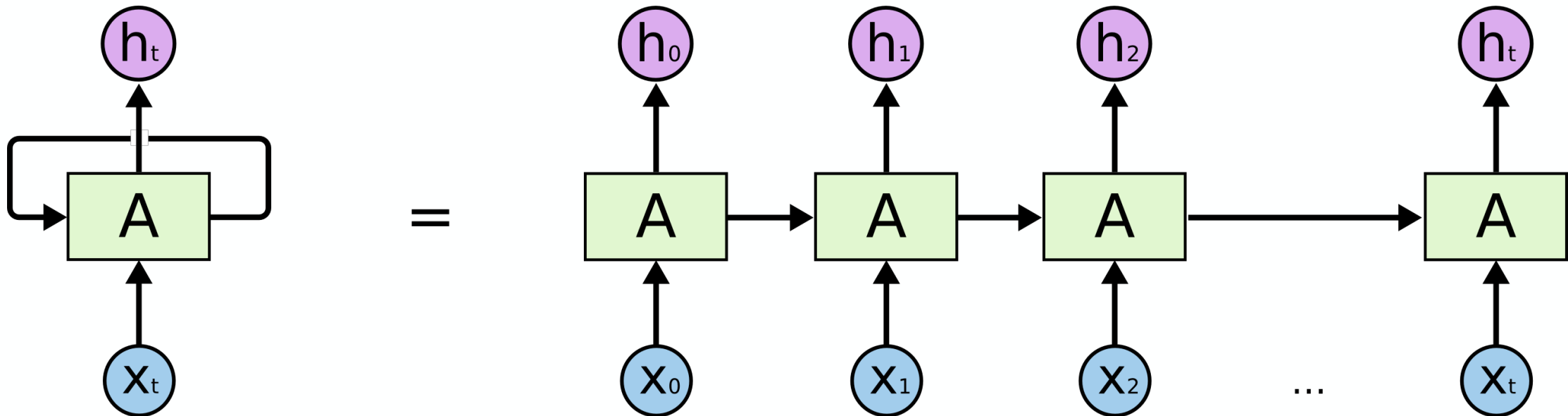


Illustration of an "unrolled" RNN.

Recurrent Neural Networks

Partner discussion

How many weight matrices W and b do we have? What is the dimension of W ?

Answer: We have one each. W has the dimension (size of hidden state) $\times$ (size of hidden state + # features)

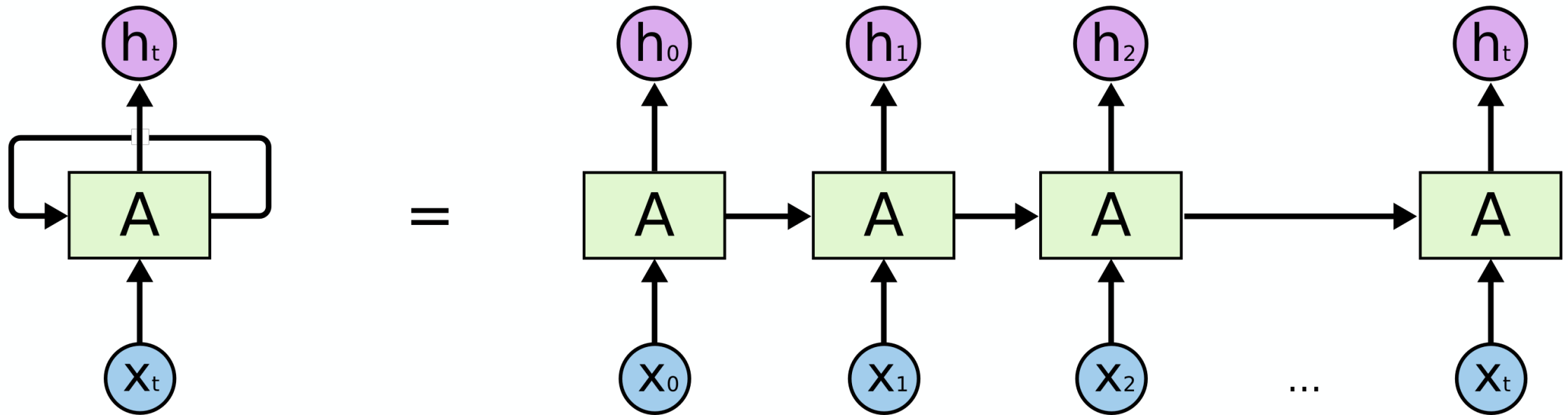
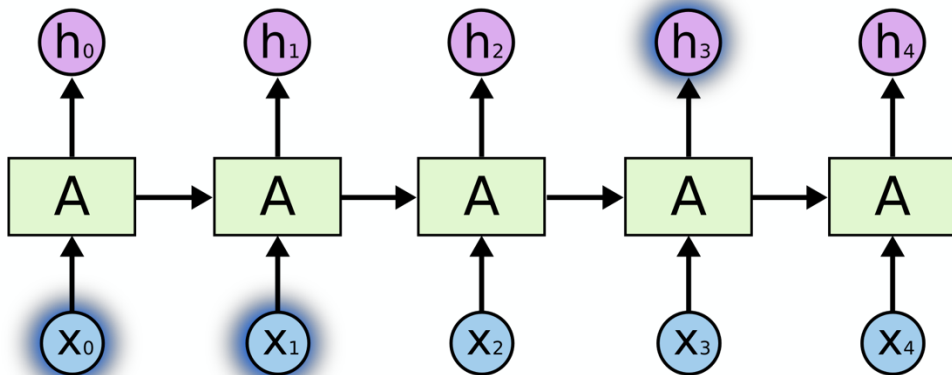


Illustration of an "unrolled" RNN.

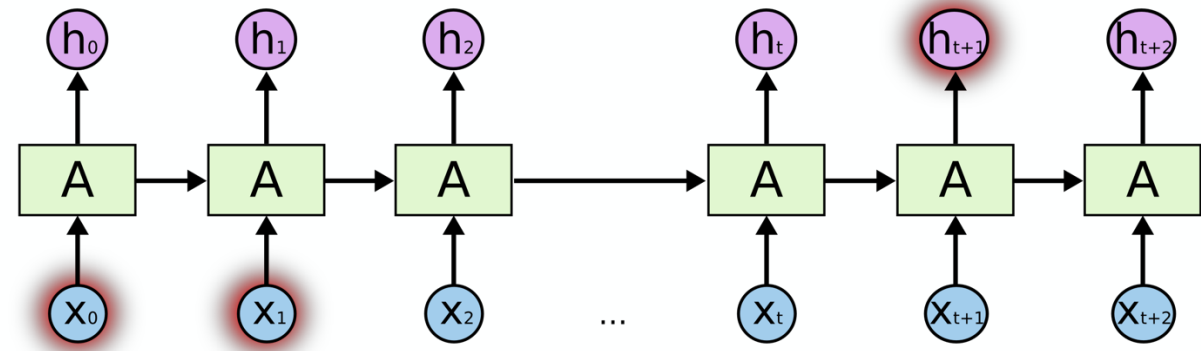
“Vanilla” Recurrent Neural Networks

Standard RNN can model short-term contexts easily. However, for sequences, the model becomes relatively deep.

“the clouds are in the *sky*”



“I grew up in France... I speak fluent *French*.”



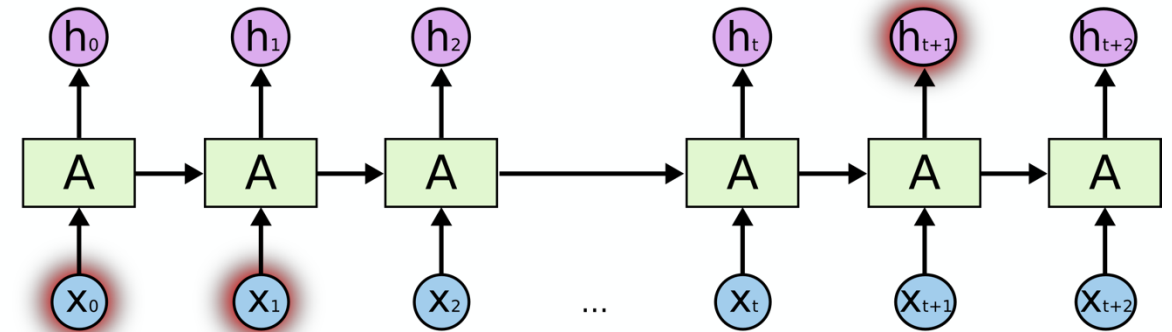
“Vanilla” Recurrent Neural Networks

For instance, given the definition:

$$h_t = A(h_{t-1}, x_t)$$
$$h_t = \tanh(W \cdot [h_{t-1}, x_t] + b)$$

Consider only three steps in:

$$h_3 = A(A(A(h_0, x_1), x_2), x_3)$$



This leads to **vanishing/exploding gradients** in model training!

- Hence, only **short-term memory** can be implemented with “vanilla” RNN.
- Therefore, never really worked well for practical applications (fell into the “**AI winter**”).

Recurrent Neural Networks

Excursion: Output layers and vector notation

- A hidden state is the hidden layer output
- Hidden layer in fully connected neural net:
 $H^{[2]} = \phi(H^{[1]}W^{[2]}) = \phi(\phi(XW^{[1]})W^{[2]})$
- Input batch size n , $X \in \mathbb{R}^{n \times d}$
- Hidden state in RNN $H_t \in \mathbb{R}^{n \times h}$:
$$H_t = A(X_t, H_{t-1})$$
$$= \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$$
- Hidden states can be used for predictions in the output layer
- Output layers have new weights and biases
 $O_t = H_t W_{hq} + b_q$

Compare:

$$h_t = \tanh(W \cdot [h_{t-1}, x_t] + b)$$

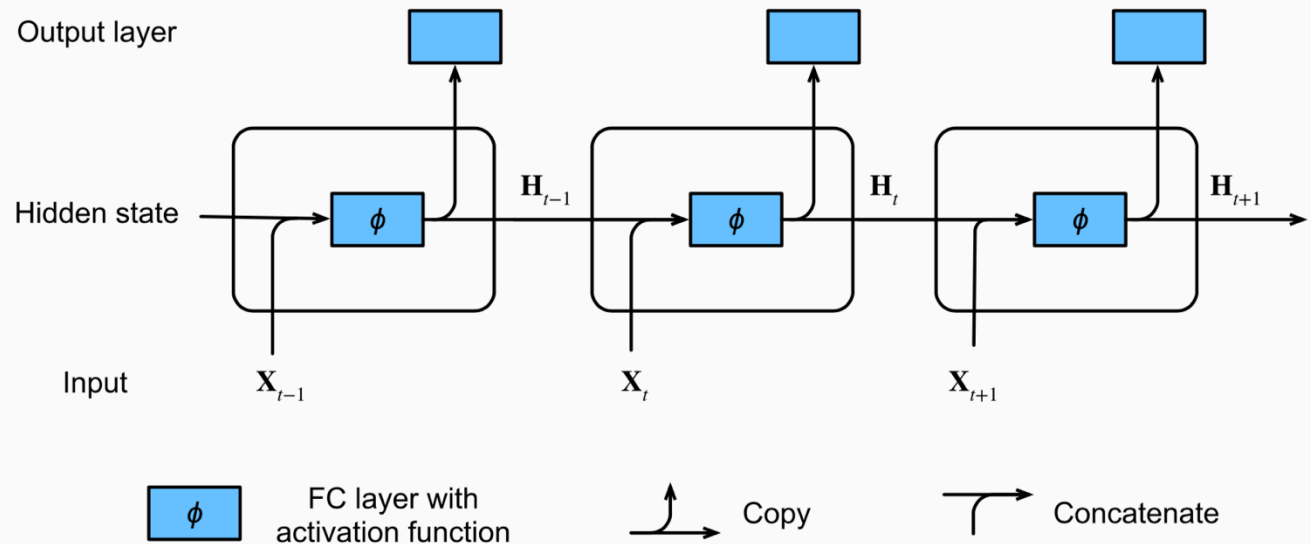


Fig. 9.4.1 An RNN with a hidden state.

Recurrent Neural Networks

Excursion: Backpropagation through time

- Simple example with linear activation functions
- Hidden state in RNN for one data point $h_t \in \mathbb{R}^h$ and output $o_t \in \mathbb{R}^q$:

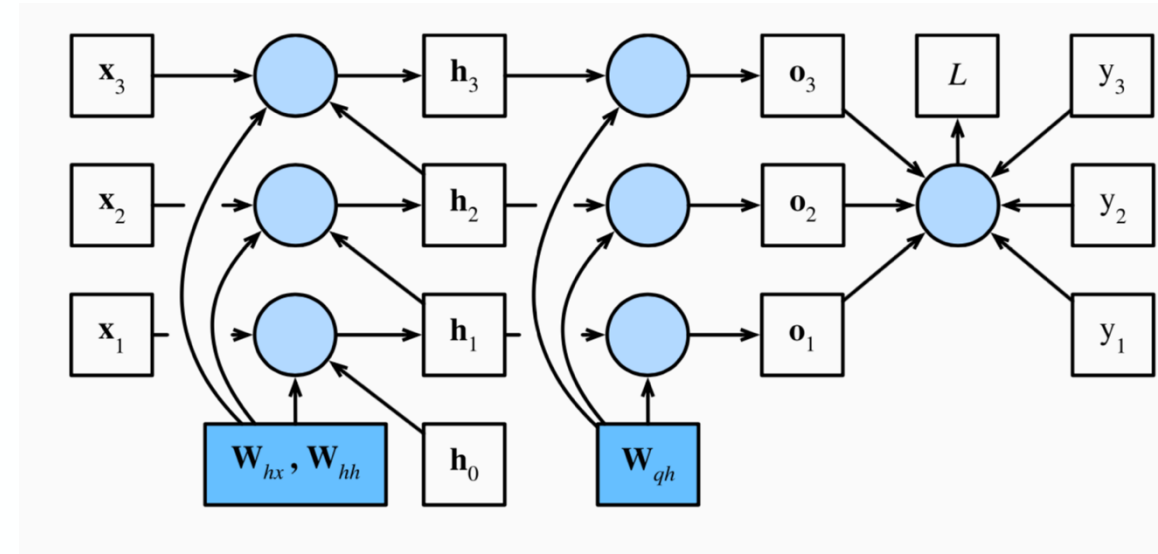
$$h_t = W_{hx}x_t + W_{hh}h_{t-1}$$

$$o_t = W_{qh}h_t$$

- Backpropagation: unrolled RNN with chain rule
- Problem: every hidden state depends again on the previous hidden state that is a function of the weights
- We have large powers of the weights in the gradient, e.g. for the output weight

$$\frac{\partial L}{\partial h_t} = \sum_{i=t}^T (W_{hh}^{Transp})^{T-i} W_{qh}^{Transp} \frac{\partial L}{\partial o_{T+t-i}}$$

- Vanishing and exploding gradients!



Computational graph showing dependencies for an RNN model with three time steps. Boxes represent variables (not shaded) or parameters (shaded) and circles represent operators.

Motivation for LSTM and GRU

Customers Review 2,491



Thanos

September 2018

Verified Purchase

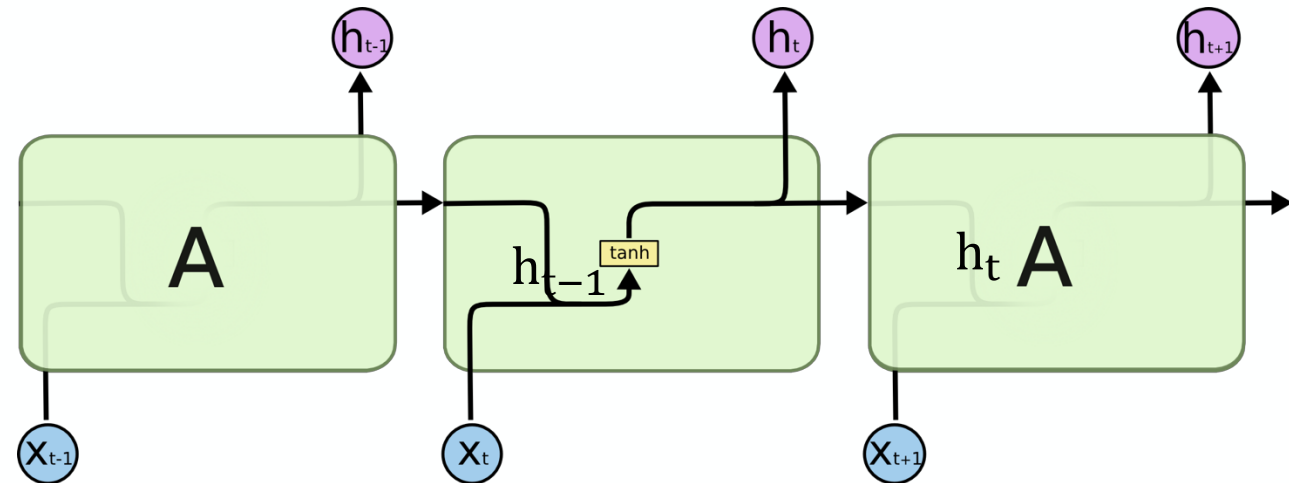
Amazing! This box of cereal gave me a perfectly balanced breakfast, as all things should be. I only ate half of it but will definitely be buying again!



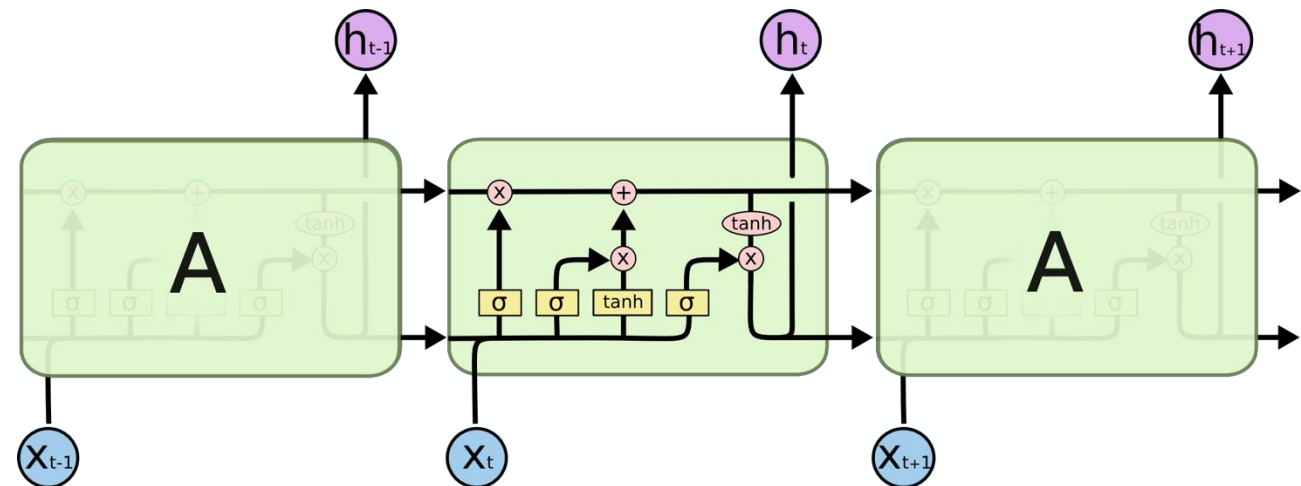
A Box of Cereal
\$3.99

Long Short Term Memory (LSTM) Walkthrough

Vanilla RNN



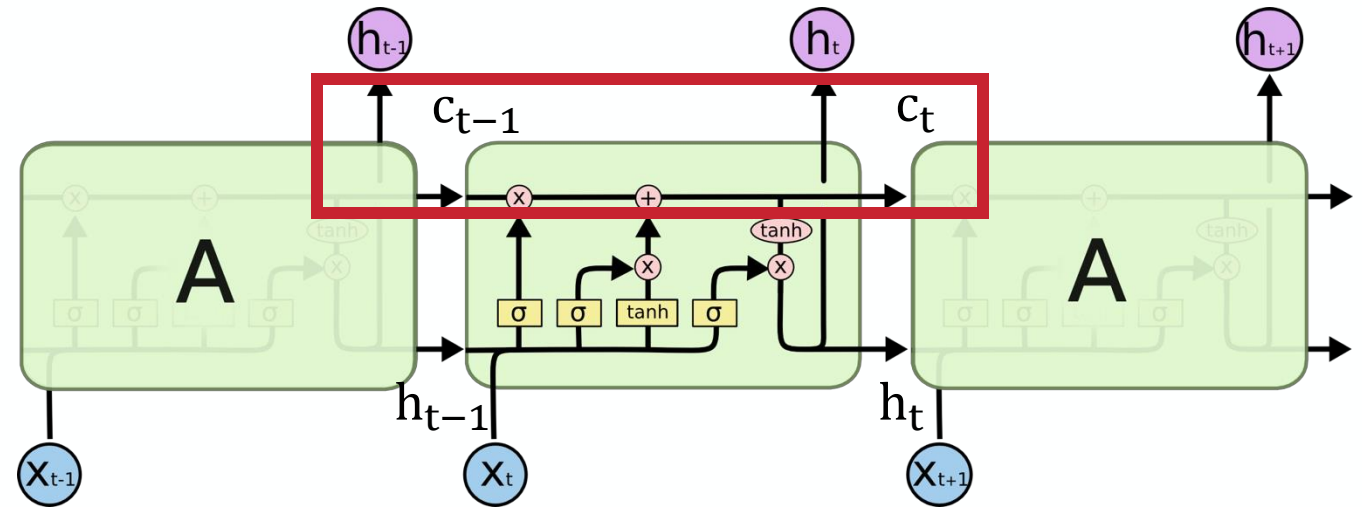
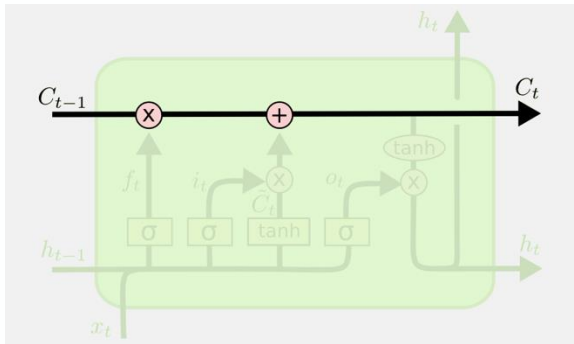
LSTM



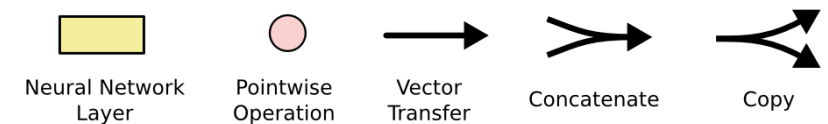
Long Short Term Memory (LSTM) Walkthrough

Now the current state depends on current value x_t , previous hidden state h_{t-1} and the previous **cell state** c_{t-1} .

Cell state brings information through the chain with minimal interaction (no weights).



Detailed illustration of the unit in an “unrolled” LSTM. Highlighting the cell state.



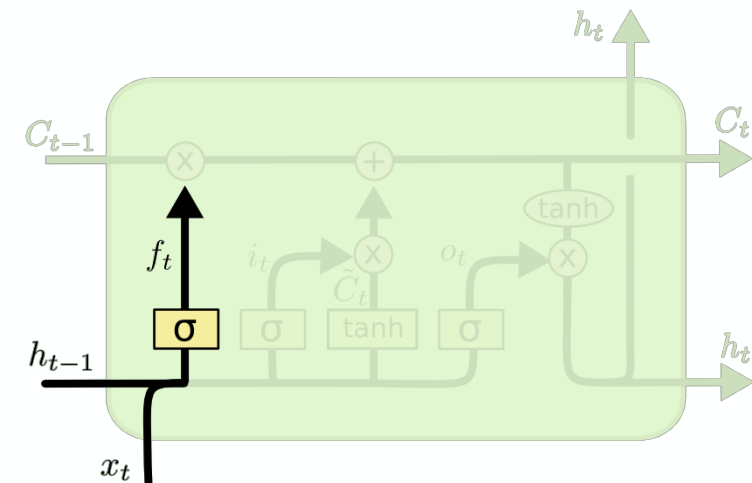
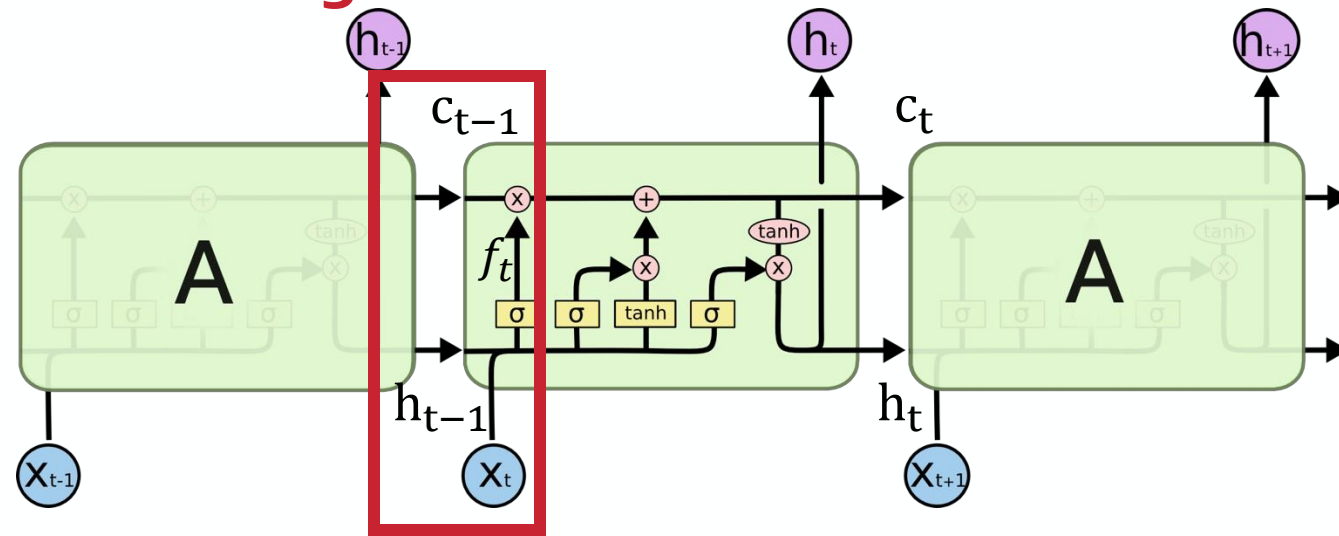
Long Short Term Memory (LSTM) Walkthrough

The first part is called the **forget gate** that adjusts the previous cell state. Note: the sigmoid activation brings this closer to zero („forgetting“) or closer to one („remembering“):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

where

- σ is the sigmoid activation function.



Detailed illustration of the forget gate.

Long Short Term Memory (LSTM) Walkthrough

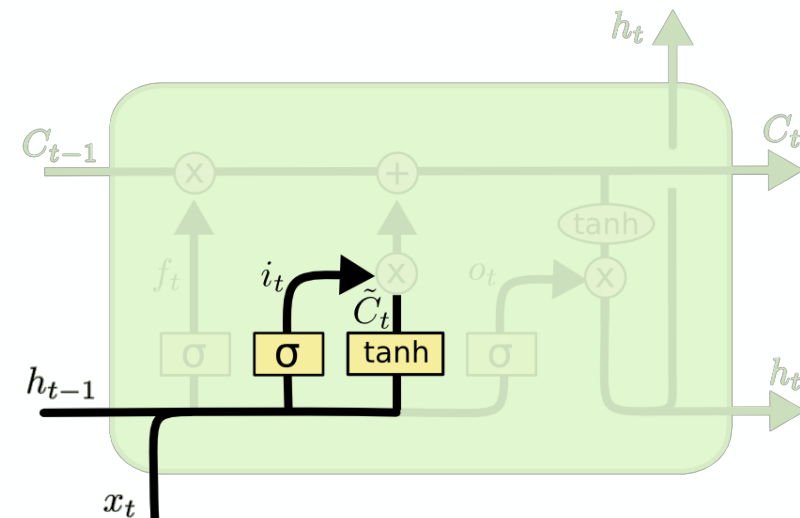
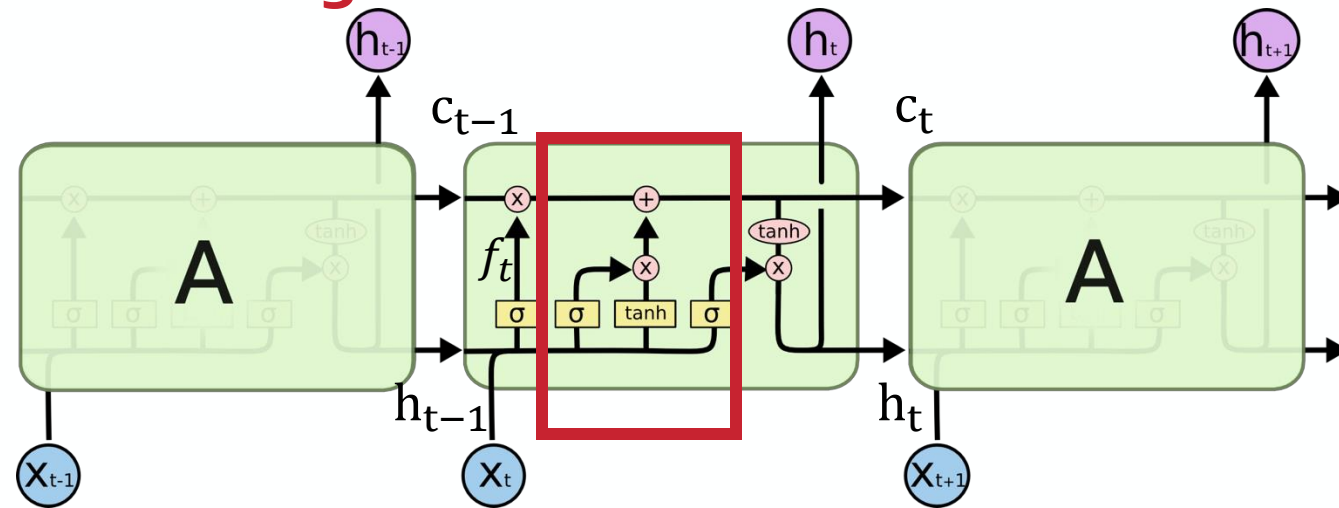
The next part is called the **input gate** that adds or subtracts to the current state. It computes input value i_t and a candidate value $\tilde{c}_t$:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{c}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Then the updated cell state c_t is computed by:

$$c_t = f_t * c_{t-1} + i_t * \tilde{c}_t$$



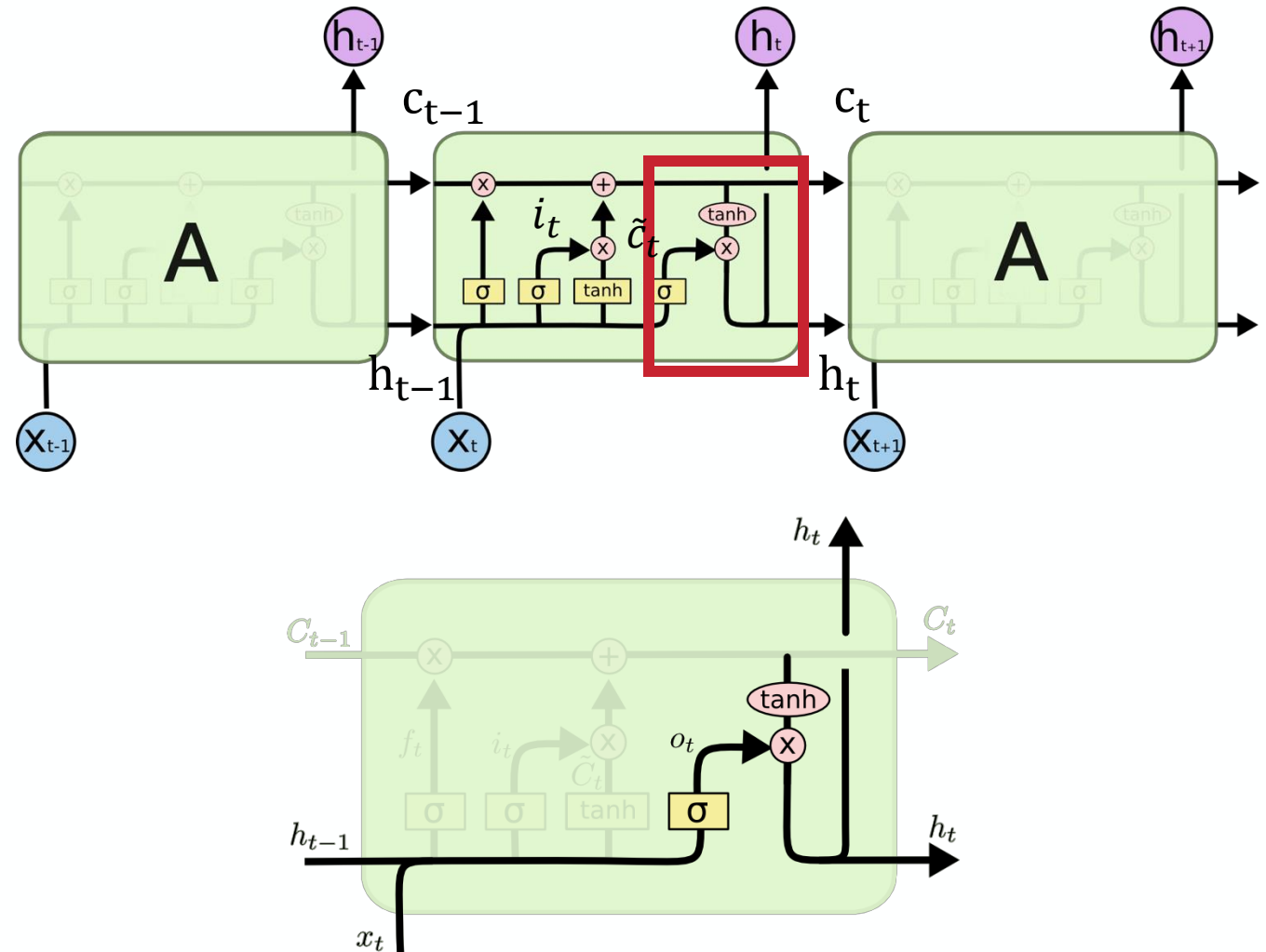
Detailed illustration of the input gate.

Long Short Term Memory (LSTM) Walkthrough

The final part is called the **output gate** that learns what to output from the cell state as the next hidden state:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(c_t)$$



Quick Gated Recurrent Units (GRU) Walkthrough

Borrow many of the ideas of LSTM:

- It combines forget and input gates into a single “update gate”,
- It merges the cell state and hidden state,
- It is simpler than the LSTM, and is hence faster to train and less prone to overfitting (e.g., when used with time series).

Update Gate:

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

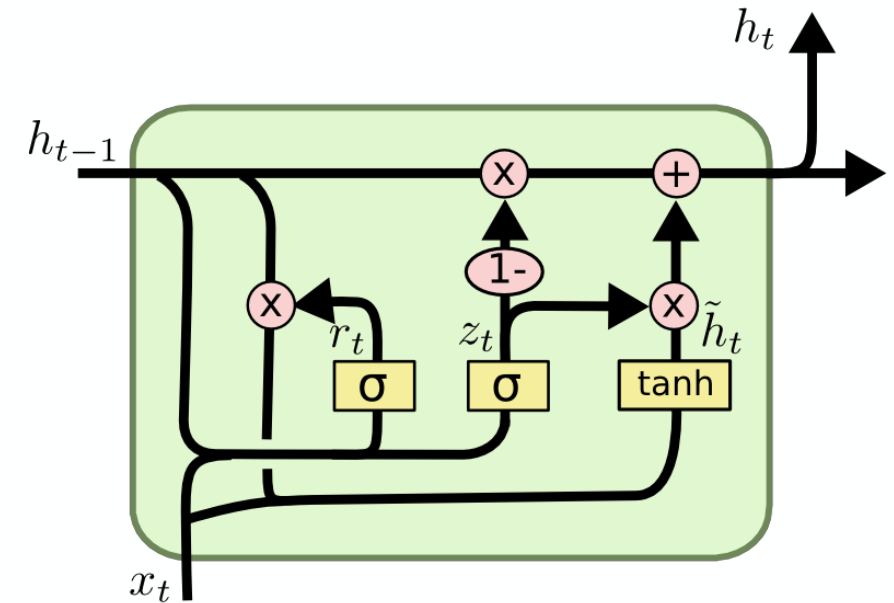
$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

Candidate hidden state:

$$\tilde{h}_t = \tanh(W_h \cdot [r_t * h_{t-1}, x_t] + b_c)$$

Final hidden state

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$



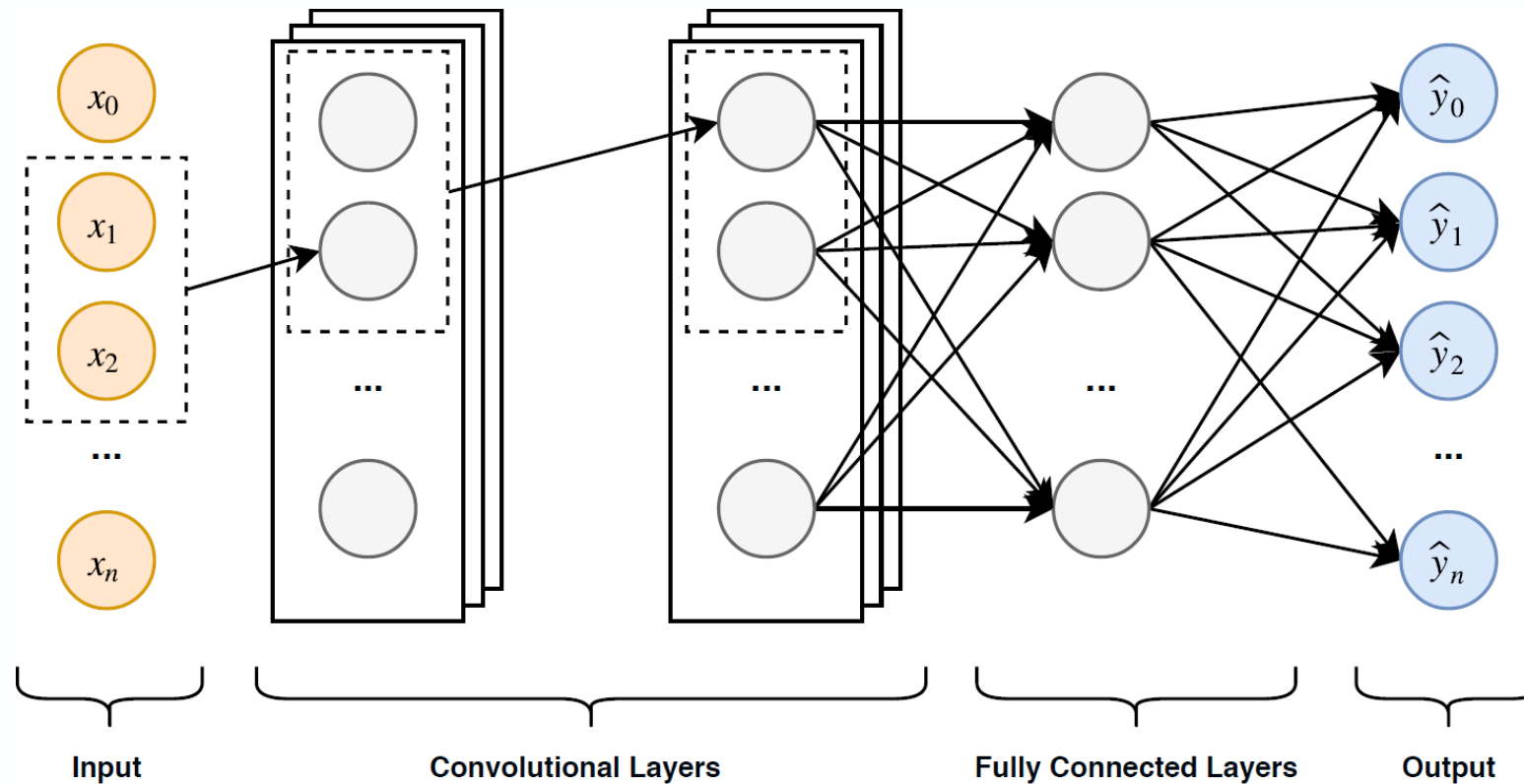
Detailed illustration of a GRU unit.

Limitations of LSTM and GRU

- Still somewhat difficult to train (for time series overfitting issues)
- They can get really deep for practical applications
 - 100 words means 100 layers
- Slow to train, as they are not easy to parallelize
- Transfer learning never really worked
- LSTMs were very popular
- Still used even though transformer models outperform these models

Convolutional Neural Networks for Sequences

CNN can be used for sequential input by feeding “windows” of data.



Convolutional Neural Networks for Sequences

Convolutions can be calculated also for 1D data:

$$h_j = (x * w)_j = \sum_{k=-p}^p x_{j+k} \cdot w_{-k} \quad , \text{ for a kernel size of } 2p+1$$

Simplified to cross-correlation:

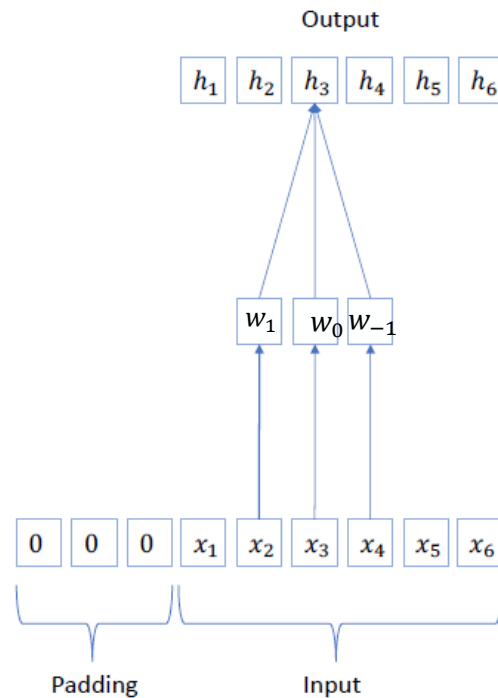
x_1	x_2	x_3	x_4	x_5	x_6
1	3	3	0	1	2

 $\ast$

w_1	w_0	w_{-1}
2	0	1

 $=$

h_2	h_3	h_4	h_5
5			



Compare 2-D:

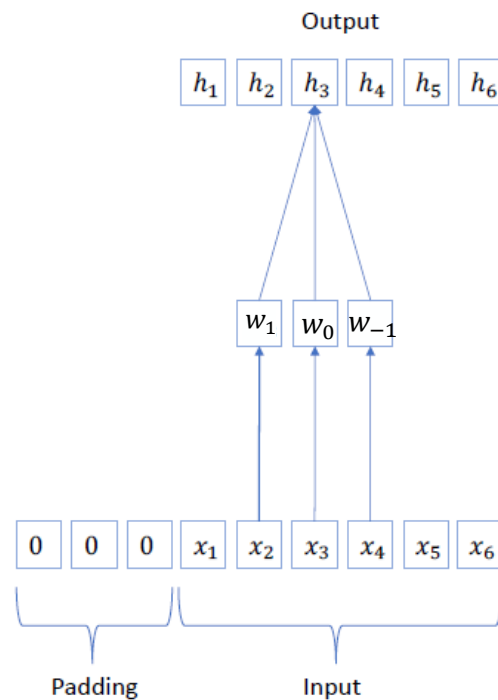
$$(X * K)_{ij} = \sum_p \sum_q x_{i+p, j+q} k_{r-p, r-q}$$

Convolutional Neural Networks for Sequences

Convolutions can be calculated also for 1D data:

$$h_j = \sum_{k=-p}^p x_{j+k} \cdot w_{-k} \quad , \text{ for a kernel size of } 2p+1$$

Simplified to cross-correlation:



x_1	x_2	x_3	x_4	x_5	x_6
1	3	3	0	1	2

*

w_1	w_0	w_{-1}
2	0	1

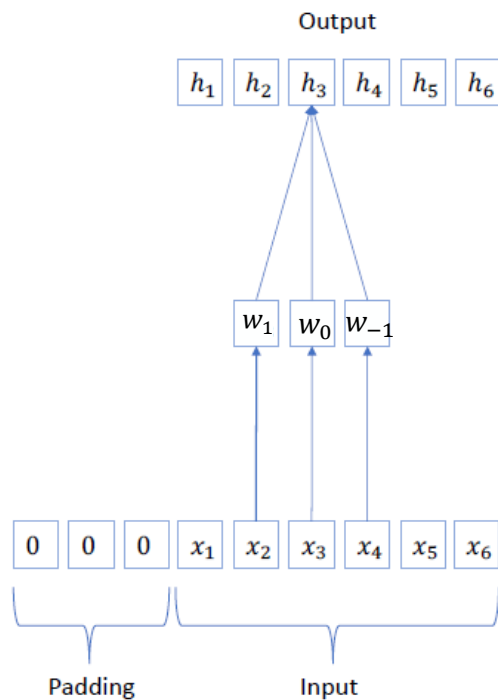
=

h_2	h_3	h_4	h_5
5	6	7	2

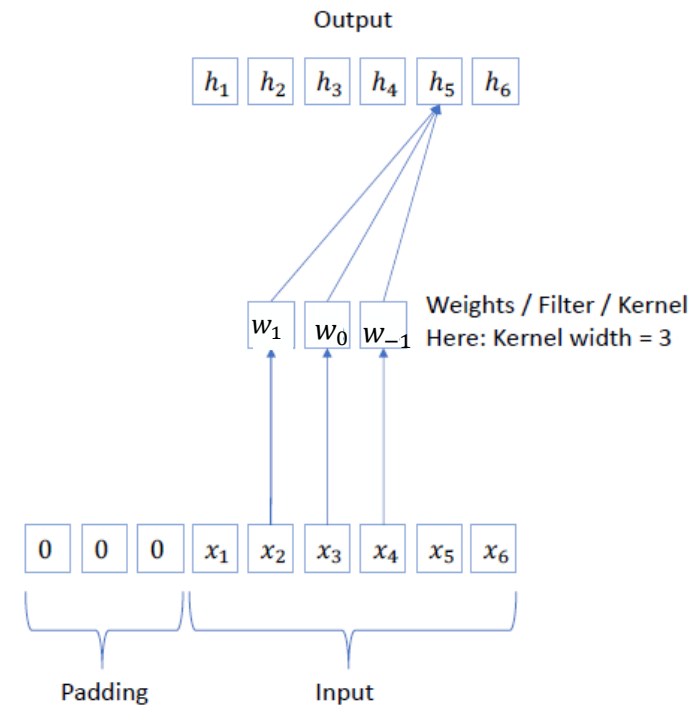
Convolutional Neural Networks for Sequences

Problem with regular convolutions: It includes future values of the time series, $h_{j+1+p} = \sum_{k=-p}^p x_{j+k} \cdot w_{-k}$, for a kernel size of $2p+1$ which can cause „leakage“ and unrealistically good forecasts.

Non-Causal Convolutions



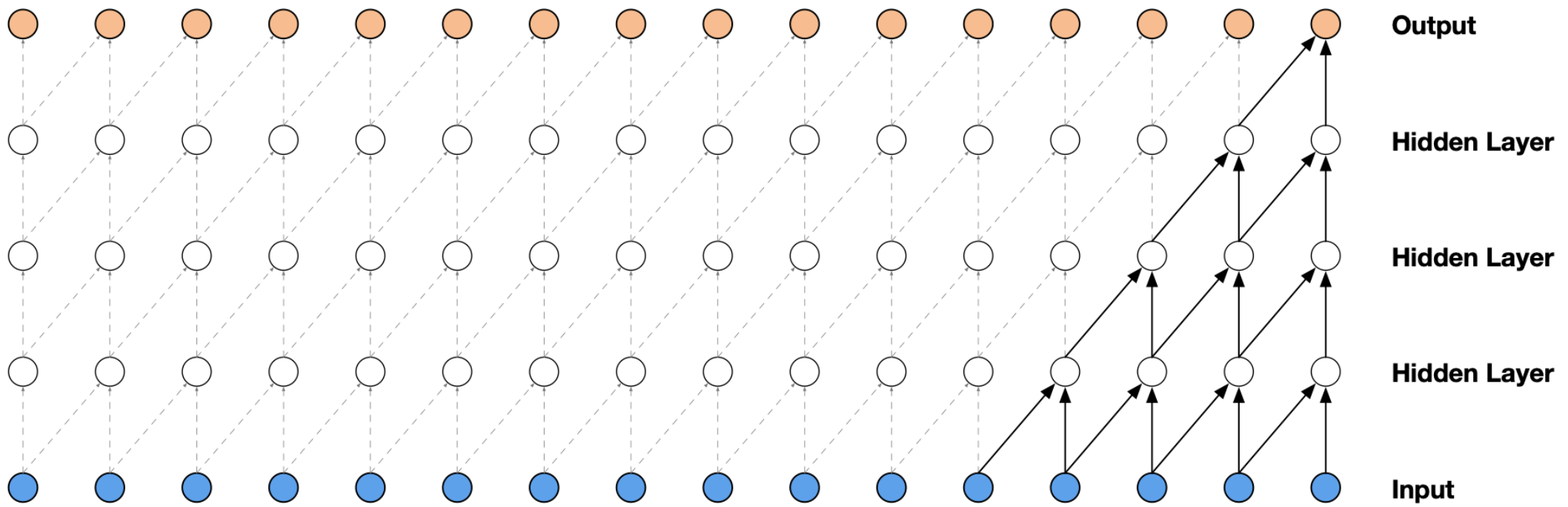
Causal Convolutions



Convolutional Neural Networks for Sequences

Second Problem with convolutions: Receptive field is limited to window length.

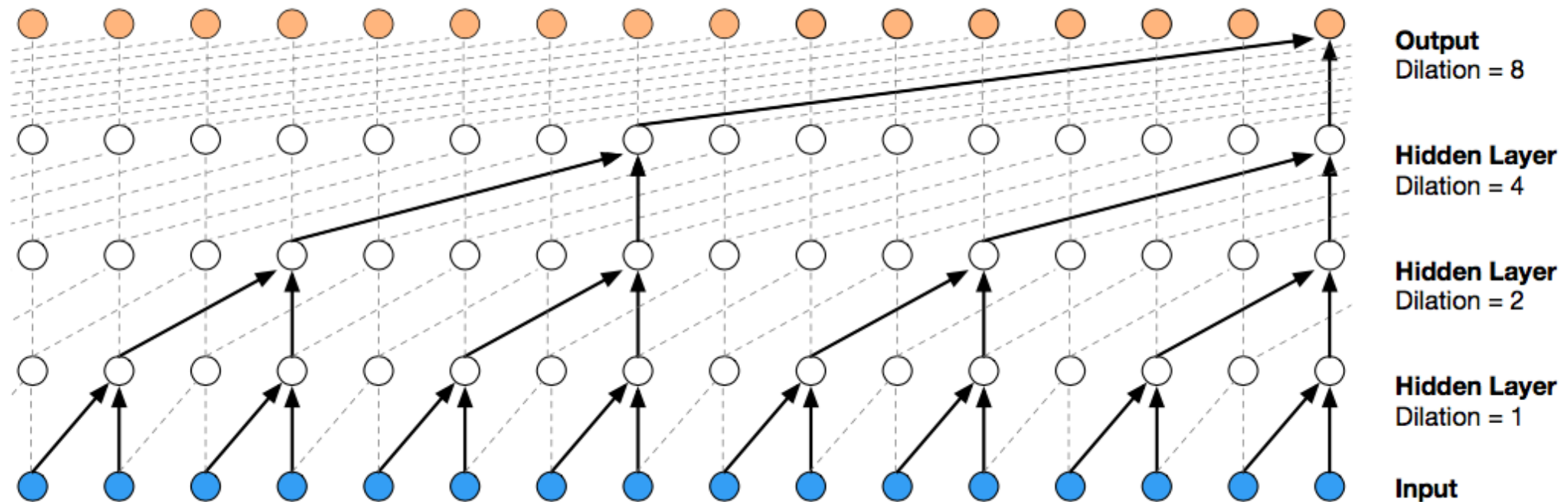
Causal Convolutions



Convolutional Neural Networks for Sequences

Second Problem with convolutions: Receptive field is limited to window length.

Dilated Convolutions



Limitations of CNN

Strengths compared to recurrent neural networks:

Can be parallized and is much faster

Weaknesses:

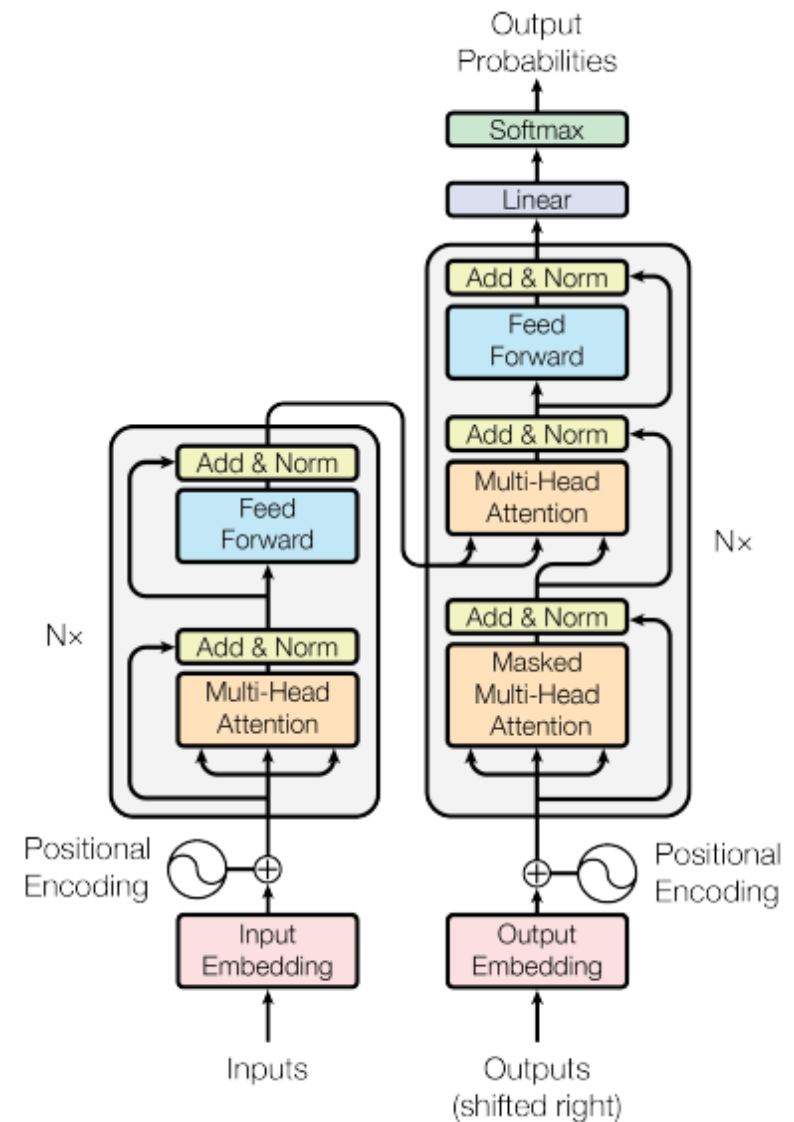
Not truly sequential, as it does not allow for different length input (only with padding).

Transformers

Is attention all you really need?

- Transformers are a model architecture that can also be considered a recurrent neural network.
- Utilizes a self-attention mechanism to draw global dependencies between input and output (related to motivation of LSTM and GRU gates).
- It's parallelizable, and scales much better than LSTM and GRU.

Will be covered in Session 8!



Example Task: Time Series Forecasting

WaveNet and Temporal Convolutional Networks (TCN)

- WaveNet was introduced for very high frequency data (sound).
- Can general speech and music
- Since then, different similar architectures have been proposed for time series and sequences more generally that rely on **dilated** and **causal convolutions** as well as regularization and skip connections (as in ResNet) that have been named **temporal convolutional networks**.

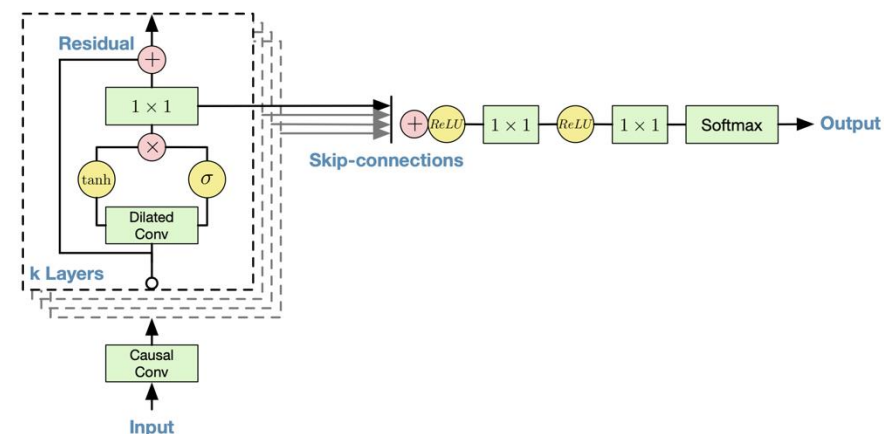


Image of WaveNet from <https://arxiv.org/pdf/1609.03499.pdf>

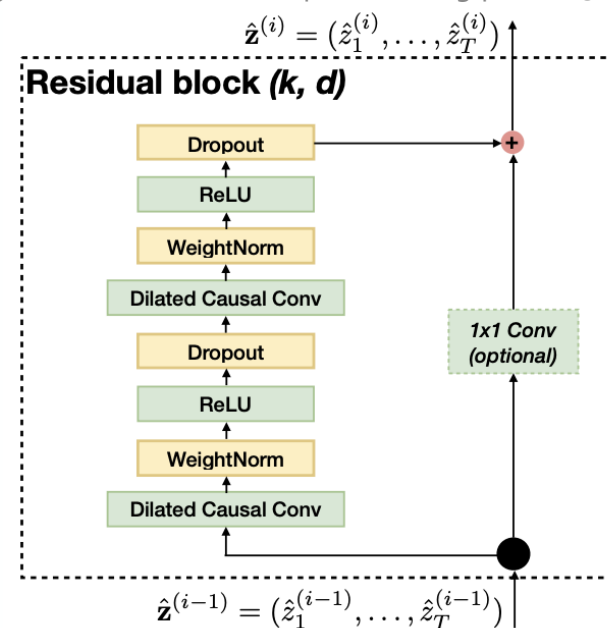
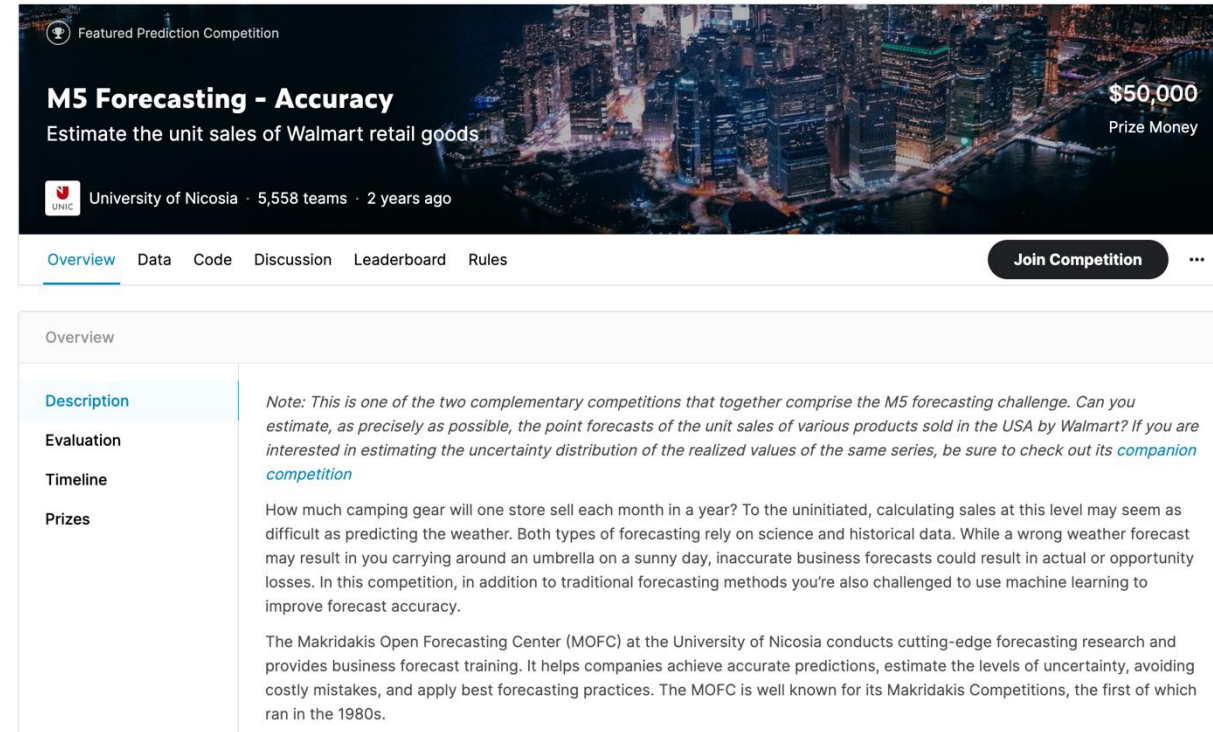


Image of TCN from <https://arxiv.org/pdf/1803.01271.pdf>

Should you use Deep Learning for Time Series Forecasting? (1/2)

- Time series forecasting has for a long time been approached by **statistical methods**, like autoregressive models (AR) and their extensions ARMA, ARIMA, and ARIMAX or models based on exponential smoothing (Holt Winters, seasonal exponential smoothing) or the Theta Method.
- Only recently machine learning models have shown to be successful in certain situations (see discussions on **forecasting competitions**).



The screenshot shows the top of a Kaggle competition page. The header features a cityscape background with the text 'Featured Prediction Competition' and 'M5 Forecasting - Accuracy'. Below this, it says 'Estimate the unit sales of Walmart retail goods' and '\$50,000 Prize Money'. The competition is hosted by the University of Nicosia, with 5,558 teams and started 2 years ago. A navigation bar includes links for Overview, Data, Code, Discussion, Leaderboard, and Rules, along with a 'Join Competition' button. The 'Overview' section is expanded, showing a description of the M5 forecasting challenge, evaluation details, a timeline, and prizes. The description notes that this is one of two complementary competitions for the M5 challenge, aimed at estimating Walmart unit sales in the USA. It also mentions the Makridakis Open Forecasting Center (MOFC) at the University of Nicosia, which provides forecasting research and training.

Screenshot of Kaggle Competition on Time Series Forecasting.

Should you use Deep Learning for Time Series Forecasting? (2/2)

When are statistical models good?

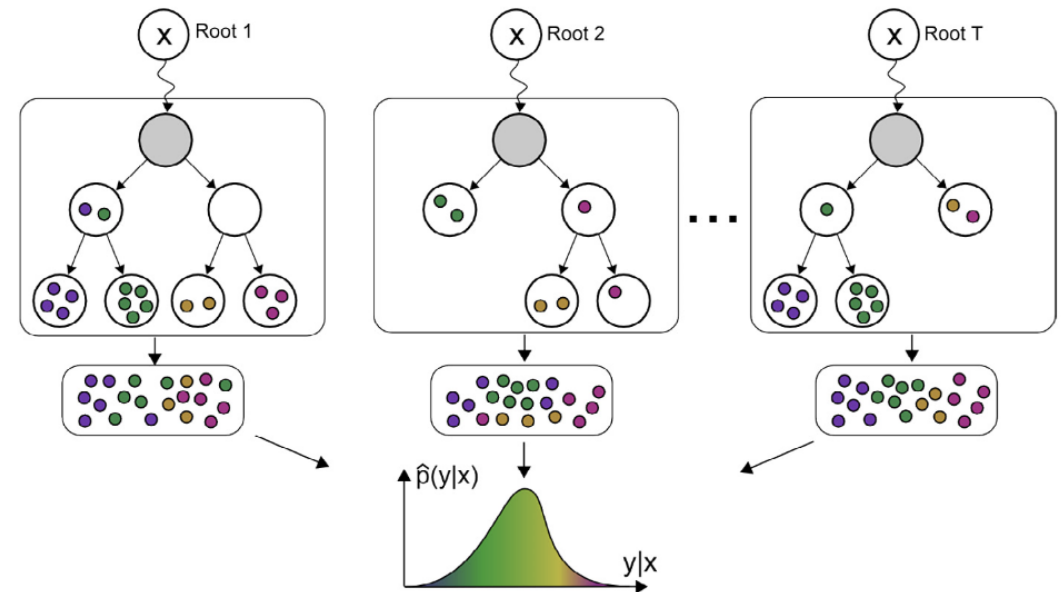
- for *local models*, i.e., when one model is fit for one specific instance, like one product type or location,
- data is of low resolution (e.g., daily, weekly, or yearly time series)
- when seasonalities and other external covariates are well understood.

When are (deep) machine learning methods good?

- fitting models across multiple similar time series (*global models*)
- hierarchical time series,
- probabilistic forecasting with complex densities (e.g. multi-modal),
- For processes with complex, non-linear external covariates and irregular seasonalities and interactions.

How to approach time series forecasting with deep machine learning models?

- Start with non-trivial benchmark models
 - Seasonal models
 - Simpler Regression models
 - Statistical Models (ARIMA, exponential smoothing, Theta)
 - For one-step ahead: compare against trend and persistence model
- Start with feature modeling and try good tabular models (e.g. random forest or gradient boosting).
- Only then implement a deep model and compare it against the benchmarks.



Illustrative: Tree Ensembles for Time Series Prediction.

- Motivating sequences and time series in public policy
- Overview of sequence modelling tasks and challenges
- Basic building blocks that comprise state-of-the-art sequence models
 - Recurrent Neural Networks (RNN)
 - Long Short Term Memory (LSTM) and Gated Recurrent Unit (GRU)
 - Temporal convolutions
- Example Task: Time series forecasting

Hertie School
Friedrichstraße 180
10117 Berlin, Germany
T +49 (0)30 259219-0
F +49 (0)30 259219-11
info@hertie-school.org
www.hertie-school.org